附录 A　支撑材料文件列表

支撑材料以 ZIP 压缩包单独提交。为便于评阅和复核，表 1仅列出文件类别、核心入口及用途；完整目录结构、运行命令、依赖版本和随机参数见压缩包根目录的 README.md。赛题提供的原始附件不重复列入自主数据资料。

表 1: 支撑材料文件列表

序号	文件或目录	内容及用途
1	README.md、各问题 README.md	总体目录、运行环境、运行顺序、输入输出与结果复现说明。
2	requirements.txt、environment.yml、config.json	Python 依赖、Conda 环境及问题一搜索参数。
3	question1/program/run.py	问题一统一运行入口：机队规模搜索、路线优化、结果输出与复核。
4	question1/program/src/uav_q1/	问题一启发式求解、精确模型、独立校验及 Excel 输出模块；完整代码见附录 B。
5	question1/results/	问题一正式工作簿、结构化路线和搜索记录。
6	question2/program/q2_solver.py	问题二负载均衡求解与独立校验程序；完整代码见附录 C。
7	question2/results/	问题二工作簿、结构化方案、汇总指标和逐架路线。
8	question3/program/q3_solver.py	问题三算法 A：直飞、必要等待与路线优化。
9	question3/program/q3_solver_enhanced.py	问题三算法 B：时变可见图、空间绕飞与两阶段优化。
10	question3/program/q3_fast_validator.py、q3_enhanced_validator.py	算法 A 快速校验与算法 B 逐事件独立复核；完整代码见附录 D。
11	question3/results/	算法 A 对照结果、算法 B 正式结果及机队规模敏感性结果。
12	generated_figures/、各问题辅助绘图与工作簿构建程序	论文图件、结果整理和格式转换文件，仅作为电子支撑材料提交。
13	AI .pdf	人工智能工具名称、用途、人工修改和核验情况。

结果口径：问题三正式方案采用算法 B，固定问题一最终采用的机队规模 $N_0 = (4, 2, 5, 4)$；算法 A 仅作基准对照，$N_0 + 1$、$N_0 + 2$ 仅用于敏感性分析。全部正式结果均由独立程序复核。验证通过仅说明方案满足本文实现的约束，不代表已经证明全局最优。

附录 B　问题一源程序

以下五个文件共同构成问题一完整程序，运行入口为 run.py。测试脚本、环境安装脚本、说明文档和预计算结果仅放入电子支撑材料。

问题一统一运行入口

路径： submission_package/question1/program/run.py

```

from __future__ import annotations

import argparse
import json
import math
import os
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent
sys.path.insert(0, str(ROOT / "src"))

# 多进程随机起点优先，避免每个工作进程再启动一组 BLAS/OpenMP 线程。
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")

import numpy as np
import pandas as pd
from scipy.optimize import linear_sum_assignment
from scipy.sparse.csgraph import minimum_spanning_tree

from uav_q1.exact_milp import result_as_dict, solve_fixed_n_exact
from uav_q1.excel_output import export_workbook
from uav_q1.solver_engine import REQ, SERVICE_H, SPEED, UNIT_KM, dist, rtime, solve
from uav_q1.validation import validate_all


def load_config() -> dict:
    return json.loads((ROOT / "config.json").read_text(encoding="utf-8"))


def pack_case(case_name: str, df: pd.DataFrame, routes: list[list[int]]) -> dict:
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    packed = []
    for uid, route in enumerate(routes, 1):
        point_ids = [int(df.iloc[i].Point_ID) for i in route]
        packed.append({
            "uav": uid,
            "sequence": [0, *point_ids, 0],
            "distance_km": dist(route, points) * UNIT_KM,
            "service_count": len(route),
            "time_h": rtime(route, points),
        })
    packed.sort(key=lambda x: x["time_h"], reverse=True)
    for uid, route in enumerate(packed, 1):
        route["uav"] = uid
    return {
        "N": len(packed),
        "Tmax": max(x["time_h"] for x in packed),
        "Tmin": min(x["time_h"] for x in packed),
        "routes": packed,
    }


def mst_lower_bound(df: pd.DataFrame, limit_h: float) -> int:
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    visits = df.Inspection_Level.map(REQ).to_numpy(int)
    augmented = np.vstack([[0.0, 0.0], points])
    matrix = np.linalg.norm(augmented[:, None, :] - augmented[None, :, :], axis=2) * UNIT_KM
    mst_km = float(minimum_spanning_tree(matrix).sum())
    total_lower_h = visits.sum() * SERVICE_H + mst_km / SPEED
    return max(1, math.ceil(total_lower_h / limit_h - 1e-12))


def _cycle_cover_lower_bound_km(df: pd.DataFrame, n_uav: int) -> float:
    """ 任务副本循环覆盖松弛；同物理点副本间以及基地副本间禁止直连。"""
    physical = np.repeat(
        np.arange(len(df), dtype=int),
        df.Inspection_Level.map(REQ).to_numpy(int),
    )
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    task_xy = points[physical]
    m = len(physical)
    size = m + int(n_uav)
    large = 1.0e12
    cost = np.full((size, size), large, dtype=float)
    task_cost = np.linalg.norm(task_xy[:, None, :] - task_xy[None, :, :], axis=2) * UNIT_KM
    task_cost[physical[:, None] == physical[None, :]] = large
    cost[:m, :m] = task_cost
    depot = np.linalg.norm(task_xy, axis=1) * UNIT_KM
    cost[:m, m:] = depot[:, None]
    cost[m:, :m] = depot[None, :]
    row, col = linear_sum_assignment(cost)
    chosen = cost[row, col]
    return math.inf if np.any(chosen >= large / 2) else float(chosen.sum())


def strong_lower_bound(df: pd.DataFrame, limit_h: float) -> dict:
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    demand = df.Inspection_Level.map(REQ).to_numpy(int)
    augmented = np.vstack([[0.0, 0.0], points])
    matrix_units = np.linalg.norm(augmented[:, None, :] - augmented[None, :, :], axis=2)
    mst_km = float(minimum_spanning_tree(matrix_units).sum()) * UNIT_KM
    repeated_degree_units = 0.0
    for idx, count in enumerate(demand, start=1):
        incident = np.delete(matrix_units[idx], idx)
        repeated_degree_units += int(count) * float(np.partition(incident, 1)[:2].sum())
    repeated_degree_km = repeated_degree_units * UNIT_KM / 2.0
    service_h = float(demand.sum()) * SERVICE_H
    farthest_h = float(np.max(2.0 * matrix_units[0, 1:] * UNIT_KM / SPEED + SERVICE_H))
    n = max(1, math.ceil(service_h / limit_h - 1e-12))
    while True:
        cycle_km = _cycle_cover_lower_bound_km(df, n)
        routing_km = max(mst_km, repeated_degree_km, cycle_km)
        aggregate_h = service_h + routing_km / SPEED
        makespan_lb = max(aggregate_h / n, farthest_h)
        if makespan_lb <= limit_h + 1e-12:
            break
        n += 1
    return {
        "N": int(n),
        "service_hours": service_h,
        "mst_km": mst_km,
        "repeated_degree_km": repeated_degree_km,
        "cycle_cover_km": cycle_km,
        "aggregate_work_hours": aggregate_h,
        "makespan_lower_bound_at_N": makespan_lb,
        "farthest_roundtrip_lower_bound_hours": farthest_h,
    }


```

```python
def routes_from_precomputed(df: pd.DataFrame, case: dict) -> list[list[int]]:
    id_to_index = {int(row.Point_ID): i for i, row in df.iterrows()}
    routes = []
    for route in case["routes"]:
        routes.append([id_to_index[int(point_id)] for point_id in route["sequence"] if int(point_id) != 0])
    return routes


def solve_from_scratch(input_path: Path, quality: str, engine: str, config: dict) -> dict:
    book = pd.ExcelFile(input_path)
    results = {}
    seeds = int(config["quality_seeds"][quality])
    for case_name in book.sheet_names:
        df = pd.read_excel(book, sheet_name=case_name)
        lower = mst_lower_bound(df, float(config["maximum_work_hours"]))
        n = lower
        limit_h = float(config["maximum_work_hours"])
        max_n = lower + int(config.get("maximum_n_increment", 6))
        history = []
        all_smaller_proven_infeasible = True
        print(f"[{case_name}] strict search starts at lower bound N={n}; quality={quality}; engine={engine}", flush=True)
        while n <= max_n:
            heuristic_routes = None
            heuristic_obj = None
            if engine in {"heuristic", "hybrid"}:
                (heuristic_obj, heuristic_routes), _, _ = solve(case_name, df, n, seeds=seeds)
                print(f"[{case_name}] N={n}, heuristic best Tmax={heuristic_obj:.6f} h", flush=True)
                record = {
                    "N": n,
                    "heuristic_best_Tmax": float(heuristic_obj),
                    "heuristic_feasible": bool(heuristic_obj <= limit_h + 1e-9),
                }
                history.append(record)
                if heuristic_obj <= limit_h + 1e-9:
                    case = pack_case(case_name, df, heuristic_routes)
                    case["theoretical_lower_bound_N"] = lower
                    case["fleet_size_status"] = (
                        "PROVEN_MINIMUM" if all_smaller_proven_infeasible else "FEASIBLE_CANDIDATE"
                    )
                    case["makespan_status"] = "BEST_FOUND_NOT_PROVEN_OPTIMAL"
                    case["search_history"] = history
                    results[case_name] = case
                    break

            if engine in {"exact", "hybrid"}:
                exact = solve_fixed_n_exact(
                    df,
                    n,
                    limit_h=limit_h,
                    time_limit_s=float(config.get("exact_time_limit_seconds", 300)),
                    mip_rel_gap=float(config.get("exact_mip_relative_gap", 0.0)),
                )
                exact_record = result_as_dict(exact)
                exact_record["N"] = n
                if history and history[-1].get("N") == n:
                    history[-1]["exact"] = exact_record
                else:
                    history.append({"N": n, "exact": exact_record})
                print(f"[{case_name}] N={n}, exact status={exact.status}, objective={exact.objective_h}", flush=True)
                if exact.status in {"OPTIMAL", "FEASIBLE"} and exact.routes is not None:
                    case = pack_case(case_name, df, exact.routes)
                    case["theoretical_lower_bound_N"] = lower
                    case["fleet_size_status"] = (
                        "PROVEN_MINIMUM" if all_smaller_proven_infeasible else "FEASIBLE_CANDIDATE"
                    )
                    case["makespan_status"] = (
                        "PROVEN_OPTIMAL" if exact.status == "OPTIMAL" else "FEASIBLE_NOT_PROVEN_OPTIMAL"
                    )
                    case["search_history"] = history
                    results[case_name] = case
                    break
                if exact.status != "INFEASIBLE":
                    # UNKNOWN 不能当作不可行；继续向上寻找只会得到可行上界。
                    all_smaller_proven_infeasible = False
                else:
                    # 启发式失败也不能当作不可行证明。
                    all_smaller_proven_infeasible = False
            n += 1
        else:
            raise RuntimeError(
                f"{case_name}: 在 N={lower}..{max_n} 内没有找到可行方案；"
                " 请提高搜索质量、精确求解时限或 maximum_n_increment"
            )
    return results


def run_ten_minute_plan(
    input_path: Path,
    config: dict,
    total_seconds: float,
    workers: int,
    candidate_k: int,
    resume_from: Path | None = None,
) -> tuple[dict, dict]:
    """ 在统一墙钟预算内验证热启动并行改善四个固定车队规模。"""
    started = time.monotonic()
    deadline = started + float(total_seconds)
    reserve = min(float(config.get("ten_minute_save_reserve_seconds", 45)), total_seconds * 0.2)
    precomputed = json.loads((ROOT / "data" / "precomputed_solution.json").read_text(encoding="utf-8"))
    validate_all(input_path, precomputed, config)
    if resume_from is not None:
        resumed = json.loads(resume_from.read_text(encoding="utf-8"))
        validate_all(input_path, resumed, config)
        precomputed = resumed
        print(f" 续搜方案独立校验通过：{resume_from}", flush=True)
    else:
        print("已有 4,2,5,4 方案独立校验通过；开始强下界与并行改进。", flush=True)

    book = pd.ExcelFile(input_path)
    weights = {k: float(v) for k, v in config.get("ten_minute_case_weights", {}).items()}
    process_order = [name for name in ["Case2", "Case4", "Case1", "Case3"] if name in book.sheet_names]
    improved: dict[str, dict] = {}
    report = {
        "mode": "ten-minute",
        "requested_total_seconds": float(total_seconds),
        "workers": int(workers),
        "candidate_neighborhood_size": int(candidate_k),
        "gpu_used": False,
        "cases": {},
    }

    for position, case_name in enumerate(process_order):
        df = pd.read_excel(book, sheet_name=case_name)
        lower = strong_lower_bound(df, float(config["maximum_work_hours"]))
        initial = precomputed[case_name]
        n_uav = int(initial["N"])
        if n_uav < int(lower["N"]):
            raise AssertionError(f"{case_name}: 已有 N={n_uav} 小于严格下界 {lower['N']}")
        warm_routes = routes_from_precomputed(df, initial)
        prior_history = list(initial.get("search_history", []))
        original_warm_tmax = float(initial.get("warm_start_Tmax", initial["Tmax"]))
```

```

        now = time.monotonic()
        usable = max(1.0, deadline - now - reserve)
        remaining_names = process_order[position:]
        denominator = sum(weights.get(name, 1.0) for name in
        ↪ remaining_names)
        budget = max(1.0, usable * weights.get(case_name, 1.0) /
        ↪ max(denominator, 1e-9))
        status = "PROVEN_MINIMUM" if n_uav == int(lower["N"]) else
        ↪ "FEASIBLE_CANDIDATE"
        print(
            f"[{case_name}] strict lower bound={lower['N']}, fixed
            ↪ N={n_uav}, "
            f"fleet status={status}, search budget={budget:.1f}s,
            ↪ workers={workers}",
            flush=True,
        )
        case_started = time.monotonic()
        (obj, routes), _, _, history = solve(
            case_name,
            df,
            n_uav,
            seeds=4096,
            time_limit_s=budget,
            workers=workers,
            warm_routes=warm_routes,
            candidate_k=candidate_k,
            progress=True,
            return_history=True,
        )
        packed = pack_case(case_name, df, routes)
        packed["theoretical_lower_bound_N"] = int(lower["N"])
        packed["lower_bound_details"] = lower
        packed["fleet_size_status"] = status
        packed["makespan_status"] = "BEST_FOUND_NOT_PROVEN_OPTIMAL"
        packed["search_history"] = prior_history + history
        packed["warm_start_Tmax"] = original_warm_tmax
        packed["improvement_hours"] = float(original_warm_tmax -
        ↪ packed["Tmax"])
        improved[case_name] = packed
        report["cases"][case_name] = {
            "N": n_uav,
            "lower_bound": lower,
            "fleet_size_status": status,
            "budget_seconds": budget,
            "actual_seconds": time.monotonic() - case_started,
            "warm_start_Tmax": original_warm_tmax,
            "resume_input_Tmax": float(initial["Tmax"]),
            "best_Tmax": float(packed["Tmax"]),
            "improvement_hours": float(original_warm_tmax -
            ↪ packed["Tmax"]),
            "completed_starts_this_run": len([x for x in history if
            ↪ x.get("seed") is not None]),
            "completed_starts": len([x for x in prior_history +
            ↪ history if x.get("seed") is not None]),
        }

    ordered = {name: improved[name] for name in book.sheet_names}
    validate_all(input_path, ordered, config)
    report["elapsed_seconds"] = time.monotonic() - started
    report["validation"] = "PASSED"
    return ordered, report


def main() -> None:
    parser = argparse.ArgumentParser(description="多无人机协同巡检问
    ↪ 题一求解器")
    parser.add_argument("--mode", choices=["reproduce", "solve",
    ↪ "ten-minute"], default="reproduce",
                        help="reproduce 复现; solve 完整搜索;
                        ↪ ten-minute 按统一短时预算并行改进")
    parser.add_argument("--quality", choices=["fast", "standard",
    ↪ "thorough"], default="standard")
    parser.add_argument("--engine", choices=["heuristic", "hybrid",
    ↪ "exact"], default="hybrid",
                        help="heuristic 仅搜索; hybrid
                        ↪ 搜索失败后精确判定; exact 仅精确 MILP")
    parser.add_argument("--total-time-limit", type=float,
    ↪ default=None, metavar="SECONDS",
                        help="ten-minute 模式的统一墙钟预算, 默认读取
                        ↪ config.json")
    parser.add_argument("--workers", type=int, default=None,
                        help="并行随机起点进程数, 默认读取
                        ↪ config.json")
    parser.add_argument("--candidate-k", type=int, default=None,
                        help=" 候选近邻数量, 默认读取 config.json")
    parser.add_argument("--input", type=Path, default=ROOT / "input"
    ↪ / "附件1.xlsx")
    parser.add_argument("--output-dir", type=Path, default=None)
    parser.add_argument("--resume-from", type=Path, default=None,
                        help="ten-minute 模式从已校验的 solution.json
                        ↪ 继续热启动搜索")
    args = parser.parse_args()

    config = load_config()
    report = None
    if args.mode == "reproduce":
        results = json.loads((ROOT / "data" /
        ↪ "precomputed_solution.json").read_text(encoding="utf-8"))
    elif args.mode == "solve":
        results = solve_from_scratch(args.input, args.quality,
        ↪ args.engine, config)
    else:
        total_seconds = float(
            args.total_time_limit
            if args.total_time_limit is not None
            else config.get("ten_minute_total_seconds", 600)
        )
        workers = int(args.workers if args.workers is not None else
        ↪ config.get("parallel_workers", 4))
        candidate_k = int(
            args.candidate_k
            if args.candidate_k is not None
            else config.get("candidate_neighborhood_size", 24)
        )
        if total_seconds <= 30 or workers < 1 or candidate_k < 1:
            parser.error("ten-minute 参数要求总时间>30 秒、workers>=1、
            ↪ candidate-k>=1")
        os.environ["LOKY_MAX_CPU_COUNT"] = str(workers)
        results, report = run_ten_minute_plan(
            args.input, config, total_seconds, workers, candidate_k,
            ↪ args.resume_from
        )

    validate_all(args.input, results, config)
    output_dir = args.output_dir or (ROOT / ("outputs_10min" if
    ↪ args.mode == "ten-minute" else "outputs"))
    output_dir.mkdir(parents=True, exist_ok=True)
    json_path = output_dir / "solution.json"
    xlsx_path = output_dir / "result1.xlsx"
    json_path.write_text(json.dumps(results, ensure_ascii=False,
    ↪ indent=2), encoding="utf-8")
    export_workbook(results, xlsx_path, config)
    if report is not None:
        (output_dir / "search_report.json").write_text(
            json.dumps(report, ensure_ascii=False, indent=2),
            ↪ encoding="utf-8"
        )
    print(f"Validation passed.\nJSON: {json_path}\nExcel:
    ↪ {xlsx_path}")


if __name__ == "__main__":
    main()
```

问题一启发式求解核心模块

路径：submission_package/question1/program/src/uav_q1/solver_engine.py

```
import math, json, os, random, time
from concurrent.futures import FIRST_COMPLETED, ProcessPoolExecutor,
↪ wait
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
```

```python
from scipy.sparse.csgraph import minimum_spanning_tree

SPEED=55.0; UNIT_KM=0.1; SERVICE_H=5/60
REQ={'I':3,'II':2,'III':1}

def valid(r):
    return all(r[i]!=r[i+1] for i in range(len(r)-1))

def dist(r,p):
    if not r:return 0.0
    q=p[r]
    return np.linalg.norm(q[0])+np.linalg.norm(q[-1])+np.linalg.norm(q[1:]-q[:-1],axis=1).sum()

def rtime(r,p): return dist(r,p)*UNIT_KM/SPEED+len(r)*SERVICE_H

def distance_matrix(p):
    q=np.vstack([[0.0,0.0],p])
    return np.linalg.norm(q[:,None,:]-q[None,:,:],axis=2)

def rtime_matrix(r,D):
    if not r:return 0.0
    z=[x+1 for x in r]
    units=D[0,z[0]]+D[z[-1],0]
    if len(z)>1:units+=sum(D[a,b] for a,b in zip(z,z[1:]))
    return float(units)*UNIT_KM/SPEED+len(r)*SERVICE_H

def candidate_sets(p,k):
    n=len(p);k=max(1,min(int(k),max(1,n-1)))
    raw=np.linalg.norm(p[:,None,:]-p[None,:,:],axis=2)
    return [set(np.argsort(raw[i])[1:k+1].tolist()) for i in range(n)]

def removal_time(r,pos,current_time,D):
    node=r[pos]+1;left=0 if pos==0 else r[pos-1]+1;right=0 if pos==len(r)-1 else r[pos+1]+1
    delta=D[left,right]-D[left,node]-D[node,right]
    return current_time+float(delta)*UNIT_KM/SPEED-SERVICE_H

def best_insert_fast(r,node,D,near):
    legal=[];preferred=[]
    for pos in range(len(r)+1):
        a=None if pos==0 else r[pos-1];b=None if pos==len(r) else r[pos]
        if a==node or b==node:continue
        ai=0 if a is None else a+1;bi=0 if b is None else b+1;ni=node+1
        delta=float(D[ai,ni]+D[ni,bi]-D[ai,bi])
        item=(delta,pos)
        legal.append(item)
        if a is None or b is None or a in near[node] or b in near[node]:preferred.append(item)
    pool=preferred or legal
    return min(pool) if pool else None

def two_opt(r,p,passes=20):
    r=list(r);n=len(r)
    for _ in range(passes):
        best=0; move=None
        for i in range(n-1):
            a=None if i==0 else r[i-1];b=r[i]
            for j in range(i+1,n):
                c=r[j];d=None if j==n-1 else r[j+1]
                # reversing preserves internal adjacency; check only new boundary pairs
                if a is not None and a==c:continue
                if d is not None and b==d:continue
                old=(np.linalg.norm(p[b]) if a is None else np.linalg.norm(p[a]-p[b]))+(np.linalg.norm(p[c]) if d is None else np.linalg.norm(p[c]-p[d]))
                new=(np.linalg.norm(p[c]) if a is None else np.linalg.norm(p[a]-p[c]))+(np.linalg.norm(p[b]) if d is None else np.linalg.norm(p[b]-p[d]))
                if new-old<best-1e-9:best=new-old;move=(i,j)
        if move is None:break
        i,j=move;r[i:j+1]=reversed(r[i:j+1])
    assert valid(r)
    return r

def insert_options(r,node,p):
    out=[]
    for pos in range(len(r)+1):
        a=None if pos==0 else r[pos-1];b=None if pos==len(r) else r[pos]
        if a==node or b==node:continue
        old=0 if a is None or b is None else np.linalg.norm(p[a]-p[b])
        new=(np.linalg.norm(p[node]) if a is None else np.linalg.norm(p[a]-p[node]))+(np.linalg.norm(p[node]) if b is None else np.linalg.norm(p[node]-p[b]))
        out.append((new-old,pos))
    return out

def can_remove(r,pos):
    return not (0<pos<len(r)-1 and r[pos-1]==r[pos+1])

def init_unique(points,req,k,seed,mode):
    n=len(points);routes=[[] for _ in range(k)]
    if mode=='kmeans':
        labels=KMeans(n_clusters=k,n_init=10,random_state=seed).fit(points,sample_weight=req).labels_
        groups=[np.where(labels==c)[0].tolist() for c in range(k)]
    else:
        ang=np.arctan2(points[:,1],points[:,0]);order=np.argsort(ang).tolist();shift=seed%n;order=order[shift:]+order[:shift]
        groups=[[] for _ in range(k)];loads=[0]*k
        for i in order:
            c=min(range(k),key=lambda z:loads[z]);groups[c].append(i);loads[c]+=req[i]
    for c,ids in enumerate(groups):
        if not ids:continue
        cur=min(ids,key=lambda i:np.linalg.norm(points[i]));routes[c]=[cur];rem=set(ids);rem.remove(cur)
        while rem:
            cur=min(rem,key=lambda j:np.linalg.norm(points[j]-points[cur]));routes[c].append(cur);rem.remove(cur)
        routes[c]=two_opt(routes[c],points)
    return routes

def add_extra_visits(routes,points,req,rng):
    extras=[]
    for i,r in enumerate(req):extras += [i]*(int(r)-1)
    extras.sort(key=lambda i:(-np.linalg.norm(points[i]),rng.random()))
    for node in extras:
        cur=[rtime(r,points) for r in routes];best=None
        for k,r in enumerate(routes):
            for delta,pos in insert_options(r,node,points):
                nt=cur[k]+delta*UNIT_KM/SPEED+SERVICE_H
                obj=max([cur[z] for z in range(len(routes)) if z!=k]+[nt])
                val=(obj,nt,delta)
                if best is None or val<best[0]:best=(val,k,pos)
        if best is None:raise RuntimeError('No valid insertion')
        _,k,pos=best;routes[k].insert(pos,node)
    return [two_opt(r,points) for r in routes]

def improve(routes,p,rng,iters,candidate_k=24):
    routes=[two_opt(r,p) for r in routes];D=distance_matrix(p);near=candidate_sets(p,candidate_k)
    times=np.array([rtime_matrix(r,D) for r in routes])
    best=[r[:] for r in routes];best_obj=times.max();temp=.06
    for it in range(iters):
        src=int(np.argmax(times)) if rng.random()<.72 else rng.randrange(len(routes))
        if not routes[src]:continue
        pos=rng.randrange(len(routes[src]));node=routes[src][pos]
        if not can_remove(routes[src],pos):continue
        nrs=routes[src][:pos]+routes[src][pos+1:];ts=removal_time(routes[src],pos,times[src],D)
        bestmove=None
        for dst in range(len(routes)):
            if dst==src:continue
            choice=best_insert_fast(routes[dst],node,D,near)
            if choice is None:continue
            delta,pos2=choice
            nrd=routes[dst][:pos2]+[node]+routes[dst][pos2:]
            td=times[dst]+delta*UNIT_KM/SPEED+SERVICE_H
            others=[times[z] for z in range(len(routes)) if z not in (src,dst)]
            newmax=max(others+[ts,td]) if others else max(ts,td)
```

```
            cand=(newmax,dst,pos2,nrd,td)
            if bestmove is None or cand[0]<bestmove[0]:bestmove=cand
        if bestmove is None:continue
        newmax,dst,pos2,nrd,td=bestmove
        change=newmax-times.max()
        if change<0 or rng.random()<math.exp(-max(0,change)/max(temp,1e-8)):
            routes[src],routes[dst]=nrs,nrd;times[src],times[dst]=ts,td
            if it%250==0:
                routes[src]=two_opt(routes[src],p,4);routes[dst]=two_opt(routes[dst],p,4)
                times[src]=rtime_matrix(routes[src],D);times[dst]=rtime_matrix(routes[dst],D)
            if times.max()<best_obj-1e-9:
                best_obj=times.max();best=[r[:] for r in routes]
        temp*=.99975
    best=[two_opt(r,p,30) for r in best]
    return polish_balance(best,p,candidate_k=candidate_k)

def polish_balance(routes,p,rounds=80,candidate_k=24):
    routes=[r[:] for r in routes];D=distance_matrix(p);near=candidate_sets(p,candidate_k)
    for _ in range(rounds):
        times=[rtime_matrix(r,D) for r in routes];old=max(times);src=int(np.argmax(times));candidate=None
        # Exhaustive relocate from current longest route.
        for pos,node in enumerate(routes[src]):
            if not can_remove(routes[src],pos):continue
            a=routes[src][:pos]+routes[src][pos+1:];ta=removal_time(routes[src],pos,times[src],D)
            for dst in range(len(routes)):
                if dst==src:continue
                choice=best_insert_fast(routes[dst],node,D,near)
                if choice is None:continue
                delta,q=choice
                b=routes[dst][:q]+[node]+routes[dst][q:];tb=times[dst]+delta*UNIT_KM/SPEED+SERVICE_H
                nm=max([times[z] for z in range(len(routes)) if z not in (src,dst)]+[ta,tb])
                if nm<old-1e-9 and (candidate is None or nm<candidate[0]):candidate=(nm,src,dst,a,b)
        # Exhaustive one-for-one swaps involving current longest route.
        for dst in range(len(routes)):
            if dst==src:continue
            for i,a_node in enumerate(routes[src]):
                for j,b_node in enumerate(routes[dst]):
                    if a_node==b_node:continue
                    if b_node not in near[a_node] and a_node not in near[b_node]:continue
                    a=routes[src][:];b=routes[dst][:];a[i],b[j]=b_node,a_node
                    if not valid(a) or not valid(b):continue
                    ta=rtime_matrix(a,D);tb=rtime_matrix(b,D)
                    nm=max([times[z] for z in range(len(routes)) if z not in (src,dst)]+[ta,tb])
                    if nm<old-1e-9 and (candidate is None or nm<candidate[0]):candidate=(nm,src,dst,a,b)
        if candidate is None:break
        _,src,dst,a,b=candidate
        routes[src]=two_opt(a,p,8);routes[dst]=two_opt(b,p,8)
    return routes

def _progress(label,done,total,best,started,active,time_limit_s=None):
    width=24;elapsed=time.monotonic()-started
    ratio=elapsed/max(.001,time_limit_s) if time_limit_s is not None else done/max(1,total)
    filled=min(width,int(width*ratio));bar='*'*filled+'·'*(width-filled)
    best_text='--' if best is None else f'{best:.5f}h'
    print(f'\r[{label}] [{bar}] {done}/{total} active={active} best={best_text} elapsed={elapsed:6.1f}s',end='',flush=True)

def _solve_one_seed(payload):
    cname,df,k,s,warm_routes,candidate_k=payload
    os.environ.setdefault('OMP_NUM_THREADS','1');os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
    p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.map(REQ).to_numpy(int)
    started=time.monotonic();rng=random.Random(9187+s)
    if warm_routes is not None and s<8:
        routes=[list(r) for r in warm_routes]
    else:
        routes=init_unique(p,req,k,301+s,'kmeans' if s%2==0 else 'sweep')
        routes=add_extra_visits(routes,p,req,rng)
    routes=improve(routes,p,random.Random(40001+s),12000+80*len(df),candidate_k=candidate_k)
    obj=max(rtime(r,p) for r in routes)
    return {'seed':s,'Tmax':float(obj),'routes':routes,'elapsed_seconds':time.monotonic()-started}

def solve(cname,df,k,seeds=14,time_limit_s=None,workers=1,warm_routes=None,
          candidate_k=24,progress=False,return_history=False):
    p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.map(REQ).to_numpy(int)
    started=time.monotonic();history=[];ans=None
    if warm_routes is not None:
        warm=[list(r) for r in warm_routes]
        warm_obj=max(rtime(r,p) for r in warm)
        ans=(warm_obj,warm);history.append({'kind':'warm_start','seed':None,'Tmax':float(warm_obj),'elapsed_seconds':0.0})
    workers=max(1,int(workers));seeds=max(1,int(seeds));deadline=None if time_limit_s is None else started+float(time_limit_s)

    if workers==1:
        for s in range(seeds):
            if deadline is not None and time.monotonic()>=deadline:break
            item=_solve_one_seed((cname,df,k,s,warm_routes,candidate_k));history.append({k:v for k,v in item.items() if k!='routes'})
            if ans is None or item['Tmax']<ans[0]:ans=(item['Tmax'],item['routes'])
            if progress:_progress(f'{cname} N={k}',len(history)-(1 if warm_routes is not None else 0),seeds,ans[0],started,0,time_limit_s)
    else:
        pending={};next_seed=0;done=0
        with ProcessPoolExecutor(max_workers=workers) as pool:
            while next_seed<seeds and len(pending)<workers:
                fut=pool.submit(_solve_one_seed,(cname,df,k,next_seed,warm_routes,candidate_k));pending[fut]=next_seed;next_seed+=1
            while pending:
                completed,_=wait(tuple(pending),timeout=.5,return_when=FIRST_COMPLETED)
                if not completed:
                    if progress:_progress(f'{cname} N={k}',done,seeds,None if ans is None else ans[0],started,len(pending),time_limit_s)
                    continue
                for fut in completed:
                    pending.pop(fut,None);item=fut.result();done+=1
                    history.append({key:value for key,value in item.items() if key!='routes'})
                    if ans is None or item['Tmax']<ans[0]:ans=(item['Tmax'],item['routes'])
                    if (deadline is None or time.monotonic()<deadline) and next_seed<seeds:
                        nxt=pool.submit(_solve_one_seed,(cname,df,k,next_seed,warm_routes,candidate_k));pending[nxt]=next_seed;next_seed+=1
                if progress:_progress(f'{cname} N={k}',done,seeds,None if ans is None else ans[0],started,len(pending),time_limit_s)
                if deadline is not None and time.monotonic()>=deadline:
                    next_seed=seeds
    if progress:print()
    if ans is None:raise RuntimeError(f'{cname} N={k}: no completed seed and no warm start')
```

```
    history.sort(key=lambda x:(x.get('seed') is None,-1 if x.get('seed') is None else x['seed']))
    if return_history:return ans,p,req,history
    return ans,p,req

def main():
    xl=pd.ExcelFile('upload/附件 1.xlsx');out={}
    for cname in xl.sheet_names:
        df=pd.read_excel(xl,sheet_name=cname);p=df[['X_Coordinate','Y_Coordinate']].to_numpy(float);req=df.Inspection_Level.map(REQ).to_numpy(int)
        aug=np.vstack([[0,0],p]);D=np.linalg.norm(aug[:,None]-aug[None,:],axis=2)*UNIT_KM
        mst=float(minimum_spanning_tree(D).sum());lb=math.ceil((req.sum()*SERVICE_H+mst/SPEED)/9-1e-12)
        print(cname,'tasks',req.sum(),'service',req.sum()*SERVICE_H,'LB',lb,flush=True)
        k=max(1,lb)
        while True:
            (obj,routes),p,req=solve(cname,df,k)
            times=[rtime(r,p) for r in routes]
            print(' k',k,'T',np.round(sorted(times),5),'valid',all(valid(r) for r in routes),flush=True)
            if max(times)<=9+1e-9:break
            k+=1
        rr=[]
        for uid,r in enumerate(routes,1):
            seq=[int(df.iloc[i].Point_ID) for i in r]
            rr.append({'uav':uid,'sequence':[0]+seq+[0],'distance_km':dist(r,p)*UNIT_KM,'service_count':len(r),'time_h':rtime(r,p)})
        rr.sort(key=lambda x:x['time_h'],reverse=True)
        for uid,x in enumerate(rr,1):x['uav']=uid
        out[cname]={'N':k,'lower_bound_N':lb,'Tmax':max(x['time_h'] for x in rr),'Tmin':min(x['time_h'] for x in rr),'routes':rr}
    json.dump(out,open('q1_results_revisit.json','w'),ensure_ascii=False,indent=2)

if __name__=='__main__':main()
```

问题一固定机队规模精确模型

路径： submission_package/question1/program/src/uav_q1/exact_milp.py

```
""" 固定无人机数量下的精确 MILP 模型。

该模块不替代快速启发式算法，而是用于区分：

* OPTIMAL：已证明固定 N 下的最优最长工作时间；
* INFEASIBLE：已证明固定 N 无法在 9 小时内完成；
* FEASIBLE：时限内得到可行解，但尚未证明最优；
* UNKNOWN：时限内既没有可行解，也没有不可行证明。

模型把每一次巡检展开成一个任务副本。同一物理点的任务副本之间不建立
直接弧，因此同一无人机不能靠原地停留重复计数，但可以在访问其他点后返回。
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any

import numpy as np
import pandas as pd
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import coo_matrix

from .solver_engine import REQ, SERVICE_H, SPEED, UNIT_KM, rtime


@dataclass
class ExactResult:
    status: str
    message: str
    routes: list[list[int]] | None
    objective_h: float | None
    dual_bound_h: float | None
    mip_gap: float | None
    node_count: int | None


class _Rows:
    """ 稀疏线性约束构造器。"""

    def __init__(self) -> None:
        self.row: list[int] = []
        self.col: list[int] = []
        self.data: list[float] = []
        self.lb: list[float] = []
        self.ub: list[float] = []

    def add(self, terms: dict[int, float], lb: float, ub: float) -> None:
        rid = len(self.lb)
        for col, value in terms.items():
            if abs(value) > 1e-15:
                self.row.append(rid)
                self.col.append(col)
                self.data.append(float(value))
        self.lb.append(float(lb))
        self.ub.append(float(ub))

    def constraint(self, nvar: int) -> LinearConstraint:
        matrix = coo_matrix(
            (self.data, (self.row, self.col)),
            shape=(len(self.lb), nvar),
        ).tocsr()
        return LinearConstraint(matrix, np.asarray(self.lb), np.asarray(self.ub))


def _task_copies(df: pd.DataFrame) -> tuple[np.ndarray, np.ndarray]:
    """ 返回每个任务副本对应的物理行号和坐标。"""
    physical: list[int] = []
    for i, level in enumerate(df["Inspection_Level"]):
        physical.extend([i] * REQ[str(level)])
    physical_arr = np.asarray(physical, dtype=int)
    points = df[["X_Coordinate", "Y_Coordinate"]].to_numpy(float)
    return physical_arr, points[physical_arr]


def solve_fixed_n_exact(
    df: pd.DataFrame,
    n_uav: int,
    limit_h: float = 9.0,
    time_limit_s: float = 300.0,
    mip_rel_gap: float = 0.0,
) -> ExactResult:
    """ 用完整弧集 MILP 求固定 ``n_uav`` 的 min-max 路径。

    SciPy/HiGHS 返回 status=2 时才记为严格不可行。达到时限且没有解时记为
    UNKNOWN，程序不得据此排除当前 N。
    """
    physical, task_xy = _task_copies(df)
    m = len(physical)
    if n_uav < 1 or n_uav > m:
        raise ValueError("n_uav 必须位于 1 和任务副本总数之间")

    # 节点 0 为基地；1..m 为任务副本。相同物理点的副本之间禁止直接转移。
    xy = np.vstack([[0.0, 0.0], task_xy])
    arcs: list[tuple[int, int]] = []
    for i in range(m + 1):
        for j in range(m + 1):
            if i == j or (i == 0 and j == 0):
                continue
            if i > 0 and j > 0 and physical[i - 1] == physical[j - 1]:
                continue
            arcs.append((i, j))
    a_count = len(arcs)
    out_arcs: list[list[int]] = [[] for _ in range(m + 1)]
    in_arcs: list[list[int]] = [[] for _ in range(m + 1)]
    for a, (i, j) in enumerate(arcs):
        out_arcs[i].append(a)
```

```
        in_arcs[j].append(a)

    # 变量块：[x 二元弧][f 连续单商品流][y 二元分配][Cmax]。
    x0 = 0
    f0 = n_uav * a_count
    y0 = f0 + n_uav * a_count
    c_idx = y0 + n_uav * m
    nvar = c_idx + 1

    def xid(k: int, a: int) -> int:
        return x0 + k * a_count + a

    def fid(k: int, a: int) -> int:
        return f0 + k * a_count + a

    def yid(k: int, i: int) -> int:
        # i 是 1..m 的任务节点号。
        return y0 + k * m + (i - 1)

    c = np.zeros(nvar)
    c[c_idx] = 1.0
    integrality = np.zeros(nvar, dtype=np.uint8)
    integrality[x0:f0] = 1
    integrality[y0:c_idx] = 1
    lower = np.zeros(nvar)
    upper = np.full(nvar, np.inf)
    upper[x0:f0] = 1.0
    upper[f0:y0] = float(m)
    upper[y0:c_idx] = 1.0
    upper[c_idx] = float(limit_h)

    rows = _Rows()

    # 每个任务副本恰由一架无人机完成。
    for i in range(1, m + 1):
        rows.add({yid(k, i): 1.0 for k in range(n_uav)}, 1.0, 1.0)

    # 每架无人机都从基地出发一次并返回一次；任务点入度=出度=分配变量。
    for k in range(n_uav):
        rows.add({xid(k, a): 1.0 for a in out_arcs[0]}, 1.0, 1.0)
        rows.add({xid(k, a): 1.0 for a in in_arcs[0]}, 1.0, 1.0)
        for i in range(1, m + 1):
            out_terms = {xid(k, a): 1.0 for a in out_arcs[i]}
            out_terms[yid(k, i)] = -1.0
            rows.add(out_terms, 0.0, 0.0)
            in_terms = {xid(k, a): 1.0 for a in in_arcs[i]}
            in_terms[yid(k, i)] = -1.0
            rows.add(in_terms, 0.0, 0.0)

    # 单商品流消除不经过基地的子回路。
    for k in range(n_uav):
        depot_terms: dict[int, float] = {}
        for a in out_arcs[0]:
            depot_terms[fid(k, a)] = depot_terms.get(fid(k, a), 0.0)
            ↪  + 1.0
        for a in in_arcs[0]:
            depot_terms[fid(k, a)] = depot_terms.get(fid(k, a), 0.0)
            ↪  - 1.0
        for i in range(1, m + 1):
            depot_terms[yid(k, i)] = -1.0
        rows.add(depot_terms, 0.0, 0.0)

        for i in range(1, m + 1):
            terms: dict[int, float] = {yid(k, i): -1.0}
            for a in in_arcs[i]:
                terms[fid(k, a)] = terms.get(fid(k, a), 0.0) + 1.0
            for a in out_arcs[i]:
                terms[fid(k, a)] = terms.get(fid(k, a), 0.0) - 1.0
            rows.add(terms, 0.0, 0.0)

        for a in range(a_count):
            rows.add({fid(k, a): 1.0, xid(k, a): -float(m)}, -np.inf,
            ↪  0.0)

    arc_time = np.asarray([
        np.linalg.norm(xy[i] - xy[j]) * UNIT_KM / SPEED for i, j in
        ↪  arcs
    ])

    def time_terms(k: int) -> dict[int, float]:
        terms = {xid(k, a): float(arc_time[a]) for a in
        ↪  range(a_count)}
        for i in range(1, m + 1):
            terms[yid(k, i)] = SERVICE_H
        return terms

    # Cmax 上界已经固定为 9 小时；同时以 Cmax 为第二阶段目标。
    all_time_terms = [time_terms(k) for k in range(n_uav)]
    for terms in all_time_terms:
        terms = dict(terms)
        terms[c_idx] = -1.0
        rows.add(terms, -np.inf, 0.0)

    # 对同质无人机按工作时长降序编号，削减车辆置换对称性。
    for k in range(n_uav - 1):
        terms = dict(all_time_terms[k + 1])
        for idx, value in all_time_terms[k].items():
            terms[idx] = terms.get(idx, 0.0) - value
        rows.add(terms, -np.inf, 0.0)

    # 同一物理点的副本无差别：按所分配的无人机编号排序，削减副本对称性。
    for p in np.unique(physical):
        copies = np.flatnonzero(physical == p) + 1
        for left, right in zip(copies[:-1], copies[1:]):
            terms: dict[int, float] = {}
            for k in range(n_uav):
                terms[yid(k, int(left))] = float(k)
                terms[yid(k, int(right))] = -float(k)
            rows.add(terms, -np.inf, 0.0)

    result = milp(
        c=c,
        integrality=integrality,
        bounds=Bounds(lower, upper),
        constraints=rows.constraint(nvar),
        options={
            "time_limit": float(time_limit_s),
            "mip_rel_gap": float(mip_rel_gap),
            "presolve": True,
        },
    )

    status_map = {0: "OPTIMAL", 1: "LIMIT", 2: "INFEASIBLE", 3:
    ↪  "UNBOUNDED", 4: "ERROR"}
    raw_status = status_map.get(int(result.status), "ERROR")
    if raw_status == "INFEASIBLE":
        return ExactResult("INFEASIBLE", result.message, None, None,
                           getattr(result, "mip_dual_bound", None),
                           getattr(result, "mip_gap", None),
                           getattr(result, "mip_node_count", None))
    if result.x is None:
        return ExactResult("UNKNOWN", result.message, None, None,
                           getattr(result, "mip_dual_bound", None),
                           getattr(result, "mip_gap", None),
                           getattr(result, "mip_node_count", None))

    vector = np.asarray(result.x)
    routes: list[list[int]] = []
    for k in range(n_uav):
        route: list[int] = []
        current = 0
        for _ in range(m + 1):
            chosen = [a for a in out_arcs[current] if vector[xid(k,
            ↪  a)] > 0.5]
            if len(chosen) != 1:
                return ExactResult("UNKNOWN", "MILP
                ↪  解的弧结构无法提取为单一路线", None,
                                   None, getattr(result,
                                   ↪  "mip_dual_bound", None),
                                   getattr(result, "mip_gap", None),
                                   getattr(result, "mip_node_count",
                                   ↪  None))
            _, nxt = arcs[chosen[0]]
            if nxt == 0:
                break
            route.append(int(physical[nxt - 1]))
            current = nxt
        else:
            return ExactResult("UNKNOWN", "路线提取超过任务节点数",
            ↪  None, None,
                               getattr(result, "mip_dual_bound",
                               ↪  None),
                               getattr(result, "mip_gap", None),
```

```
                            getattr(result, "mip_node_count",
                            ↪ None))
        routes.append(route)

    physical_points = df[["X_Coordinate",
    ↪ "Y_Coordinate"]].to_numpy(float)
    objective = max(rtime(route, physical_points) for route in
    ↪ routes)
    final_status = "OPTIMAL" if raw_status == "OPTIMAL" else
    ↪ "FEASIBLE"
    return ExactResult(final_status, result.message, routes,
    ↪ objective,
                       getattr(result, "mip_dual_bound", None),
                       getattr(result, "mip_gap", None),
                       getattr(result, "mip_node_count", None))


def result_as_dict(result: ExactResult) -> dict[str, Any]:
    def number(value: Any) -> float | int | None:
        if value is None:
            return None
        if isinstance(value, (int, np.integer)):
            return int(value)
        return float(value)

    return {
        "status": result.status,
        "message": result.message,
        "objective_h": number(result.objective_h),
        "dual_bound_h": number(result.dual_bound_h),
        "mip_gap": number(result.mip_gap),
        "node_count": number(result.node_count),
    }
```

问题一独立结果校验模块

路径：submission_package/question1/program/src/uav_q1/validation.py

```
from __future__ import annotations

from collections import Counter
from pathlib import Path
import math
import pandas as pd


def validate_all(input_path: Path, results: dict, config: dict) ->
↪ None:
    required_map = config["level_required_visits"]
    speed = float(config["speed_kmh"])
    service_h = float(config["service_minutes_per_visit"]) / 60.0
    limit_h = float(config["maximum_work_hours"])
    book = pd.ExcelFile(input_path)
    assert set(results) == set(book.sheet_names), "结果中的 Case
    ↪ 与附件 1 不一致"

    for case_name in book.sheet_names:
        df = pd.read_excel(book, sheet_name=case_name)
        expected = {int(row.Point_ID):
        ↪ int(required_map[row.Inspection_Level]) for _, row in
        ↪ df.iterrows()}
        coordinates = {int(row.Point_ID): (float(row.X_Coordinate),
        ↪ float(row.Y_Coordinate)) for _, row in df.iterrows()}
        found = Counter()
        case = results[case_name]
        assert int(case["N"]) == len(case["routes"]), f"{case_name}:
        ↪ N 与路线行数不符"

        recalculated_times = []
        for route in case["routes"]:
            sequence = [int(x) for x in route["sequence"]]
            assert sequence[0] == sequence[-1] == 0, f"{case_name}:
            ↪ 路线必须以基地 0 为首尾"
            tasks = sequence[1:-1]
            assert all(tasks[i] != tasks[i + 1] for i in
            ↪ range(len(tasks) - 1)), \
                f"{case_name}: 同一点连续出现，未满足离开后返回规则"
            found.update(tasks)
            xy = [(0.0, 0.0), *[coordinates[x] for x in tasks], (0.0,
            ↪ 0.0)]
            coordinate_distance = sum(math.dist(xy[i], xy[i + 1]) for
            ↪ i in range(len(xy) - 1))
            distance_km = coordinate_distance *
            ↪ float(config["coordinate_unit_km"])
            work_h = distance_km / speed + len(tasks) * service_h
            assert abs(work_h - float(route["time_h"])) < 1e-6,
            ↪ f"{case_name}: 工作时间计算不一致"
            assert work_h <= limit_h + 1e-8, f"{case_name}: 存在超过 9
            ↪ h 的路线"
            recalculated_times.append(work_h)

        assert dict(found) == expected, f"{case_name}: 巡检次数不满足
        ↪ I/II/III=3/2/1"
        assert abs(max(recalculated_times) - float(case["Tmax"])) <
        ↪ 1e-6
        assert abs(min(recalculated_times) - float(case["Tmin"])) <
        ↪ 1e-6
```

问题一工作簿输出模块

路径：submission_package/question1/program/src/uav_q1/excel_output.py

```
from __future__ import annotations

from pathlib import Path
from openpyxl import Workbook
from openpyxl.styles import Alignment, Border, Font, PatternFill,
↪ Side
from openpyxl.utils import get_column_letter


NAVY = "17365D"
BLUE = "4472C4"
LIGHT = "D9EAF7"
WHITE = "FFFFFF"
THIN = Side(style="thin", color="D9E2F3")


def style_header(cells) -> None:
    for cell in cells:
        cell.fill = PatternFill("solid", fgColor=BLUE)
        cell.font = Font(bold=True, color=WHITE)
        cell.alignment = Alignment(horizontal="center",
        ↪ vertical="center", wrap_text=True)
        cell.border = Border(left=THIN, right=THIN, top=THIN,
        ↪ bottom=THIN)


def export_workbook(results: dict, output_path: Path, config: dict)
↪ -> None:
    wb = Workbook()
    ws = wb.active
    ws.title = "Summary"
    ws.sheet_view.showGridLines = False
    ws.merge_cells("A1:D1")
    ws["A1"] = " 问题一：多无人机协同巡检结果"
    ws["A1"].fill = PatternFill("solid", fgColor=NAVY)
    ws["A1"].font = Font(bold=True, color=WHITE, size=16)
    ws["A1"].alignment = Alignment(horizontal="center")
    headers = ["测试算例", "无人机数量 N", "最长工作时间 Tmax (h)",
    ↪ "最短工作时间 Tmin (h)"]
    for col, value in enumerate(headers, 1):
        ws.cell(3, col, value)
    style_header(ws[3])
    for row, (case_name, case) in enumerate(results.items(), 4):
        ws.cell(row, 1, case_name)
        ws.cell(row, 2, case["N"])
        ws.cell(row, 3, case["Tmax"])
        ws.cell(row, 4, case["Tmin"])
```

```
        ws.cell(row, 3).number_format = ws.cell(row, 4).number_format = "0.0000"
    ws["A10"] = " 最终计数规则"
    ws["B10"] = "到达并作业 5 min 计 1 次；原地停留不重复计数；离开后返回可再次计数；不同无人机可同时巡检。"
    ws.merge_cells("B10:D11")
    ws["A12"] = " 时间公式"
    ws["B12"] = "T = Flight Distance / 55 + Inspection Count × 5 / 60"
    ws.merge_cells("B12:D12")
    for col, width in zip("ABCD", [20, 25, 30, 30]):
        ws.column_dimensions[col].width = width
    ws.freeze_panes = "A4"

    for case_name, case in results.items():
        sh = wb.create_sheet(case_name)
        sh.sheet_view.showGridLines = False
        max_len = max(len(x["sequence"]) for x in case["routes"])
        headers = ["UAV ID", *[f"{i}th Inspection Point" for i in range(1, max_len + 1)],
                   "Flight Distance (km)", "Inspection Count", "Working Time (h)"]
        for col, value in enumerate(headers, 1):
            sh.cell(5, col, value)
        style_header(sh[5])
        sh["A1"] = f"{case_name} 详细调度方案（0 表示基地）"
        end_col = get_column_letter(len(headers))
        sh.merge_cells(f"A1:{end_col}1")
        sh["A1"].fill = PatternFill("solid", fgColor=NAVY)
        sh["A1"].font = Font(bold=True, color=WHITE, size=14)
        sh["A1"].alignment = Alignment(horizontal="center")
        sh["A2"] = "N"; sh["B2"] = case["N"]
        sh["C2"] = "Tmax (h)"; sh["D2"] = case["Tmax"]
        sh["E2"] = "Tmin (h)"; sh["F2"] = case["Tmin"]
        for row, route in enumerate(case["routes"], 6):
            values = [route["uav"], *route["sequence"], *([None] * (max_len - len(route["sequence"]))) ,
                      route["distance_km"], route["service_count"]]
            for col, value in enumerate(values, 1): sh.cell(row, col, value)
            distance_col = 2 + max_len
            count_col = distance_col + 1
            time_col = distance_col + 2
            sh.cell(row, time_col,
            f"={get_column_letter(distance_col)}{row}/55+{get_column_letter(count_col)}{row}*5/60")
            sh.cell(row, distance_col).number_format = "0.000"
            sh.cell(row, time_col).number_format = "0.000000"
        sh.freeze_panes = "B6"
        sh.column_dimensions["A"].width = 10
        for c in range(2, 2 + max_len):
            sh.column_dimensions[get_column_letter(c)].width = 11
        for c in range(2 + max_len, len(headers) + 1):
            sh.column_dimensions[get_column_letter(c)].width = 19
        sh.auto_filter.ref = f"A5:{end_col}{5 + len(case['routes'])}"

    output_path.parent.mkdir(parents=True, exist_ok=True)
    wb.save(output_path)
```

附录 C　问题二源程序

问题二求解、结果复算及 CSV 输出均集中在同一程序中；用于生成论文图件和最终 Excel 版式的辅助脚本仅放入电子支撑材料。

问题二负载均衡求解与校验程序

路径：submission_package/question2/program/q2_solver.py

```python
""" 问题二：固定问题一无人机数量与完成时间上界的负载均衡优化。

Python 求解器只输出 JSON/CSV；result2.xlsx 由 build_result2.mjs 根据同一 JSON 生成。
工作时间仅包含实际飞行与有效巡检服务，不允许用等待时间改善均衡指标。
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import random
import time
from collections import Counter
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd


SPEED_KMH = 55.0
UNIT_KM = 0.1
SERVICE_H = 5.0 / 60.0
REQ = {"I": 3, "II": 2, "III": 1}
TOL = 1.0e-9


@dataclass(frozen=True)
class CaseData:
    point_ids: tuple[int, ...]
    coordinates: dict[int, tuple[float, float]]
    expected_visits: dict[int, int]
    distance_km: dict[tuple[int, int], float]


def load_case(book: pd.ExcelFile, case_name: str) -> CaseData:
    df = pd.read_excel(book, sheet_name=case_name)
    ids = tuple(int(x) for x in df["Point_ID"])
    coordinates = {
        int(row.Point_ID): (float(row.X_Coordinate), float(row.Y_Coordinate))
        for _, row in df.iterrows()
    }
    coordinates[0] = (0.0, 0.0)
    expected = {
        int(row.Point_ID): REQ[str(row.Inspection_Level)]
        for _, row in df.iterrows()
    }
    distance: dict[tuple[int, int], float] = {}
    for i in (0,) + ids:
        xi, yi = coordinates[i]
        for j in (0,) + ids:
            xj, yj = coordinates[j]
            distance[(i, j)] = math.hypot(xi - xj, yi - yj) * UNIT_KM
    return CaseData(ids, coordinates, expected, distance)


def route_valid(route: list[int]) -> bool:
    return bool(route) and all(a != b for a, b in zip(route, route[1:]))


def route_metrics(route: list[int], data: CaseData) -> tuple[float, float]:
    seq = [0] + route + [0]
    distance = sum(data.distance_km[(a, b)] for a, b in zip(seq, seq[1:]))
    return distance, distance / SPEED_KMH + len(route) * SERVICE_H


def all_metrics(routes: list[list[int]], data: CaseData) -> tuple[list[float], list[float]]:
    pairs = [route_metrics(route, data) for route in routes]
    return [x[0] for x in pairs], [x[1] for x in pairs]


def objective(times_h: list[float]) -> tuple[float, float, float]:
    return max(times_h) - min(times_h), max(times_h), sum(times_h)


def better(a: tuple[float, float, float], b: tuple[float, float, float]) -> bool:
    for x, y in zip(a, b):
        if x < y - TOL:
            return True
        if x > y + TOL:
            return False
    return False


def two_opt(route: list[int], data: CaseData, max_passes: int = 2) -> list[int]:
    """ 仅执行缩短飞行距离的合法 2-opt，不会人为延长轻载路线。"""
    best = route[:]
    if len(best) < 4:
        return best
    for _ in range(max_passes):
        changed = False
        for i in range(len(best) - 2):
            a = 0 if i == 0 else best[i - 1]
            b = best[i]
            for j in range(i + 1, len(best)):
                c = best[j]
                d = 0 if j == len(best) - 1 else best[j + 1]
                old = data.distance_km[(a, b)] + data.distance_km[(c, d)]
                new = data.distance_km[(a, c)] + data.distance_km[(b, d)]
                if new >= old - 1.0e-12:
                    continue
                candidate = best[:i] + list(reversed(best[i : j + 1])) + best[j + 1 :]
                if route_valid(candidate):
                    best = candidate
                    changed = True
                    break
            if changed:
                break
        if not changed:
            break
    return best


def normalize_changed(
    routes: list[list[int]], data: CaseData, changed: tuple[int, ...]
) -> list[list[int]]:
    result = [r[:] for r in routes]
    for idx in set(changed):
        result[idx] = two_opt(result[idx], data, max_passes=2)
    return result


def legal_remove(route: list[int], pos: int) -> bool:
```

```
    if len(route) <= 1:
        return False
    candidate = route[:pos] + route[pos + 1 :]
    return route_valid(candidate)


def legal_insert(route: list[int], node: int, pos: int) -> bool:
    left = None if pos == 0 else route[pos - 1]
    right = None if pos == len(route) else route[pos]
    return left != node and right != node


def choose_heavy_light(times_h: list[float], rng: random.Random) -> tuple[int, int]:
    order = sorted(range(len(times_h)), key=times_h.__getitem__)
    width = min(2, max(1, len(order) // 2))
    if rng.random() < 0.85:
        src = rng.choice(order[-width:])
        dst = rng.choice(order[:width])
    else:
        src, dst = rng.sample(order, 2)
    return src, dst


def propose(
    routes: list[list[int]], data: CaseData, rng: random.Random
) -> list[list[int]] | None:
    _, times = all_metrics(routes, data)
    src, dst = choose_heavy_light(times, rng)
    move = rng.random()
    candidate = [r[:] for r in routes]

    if move < 0.60:
        positions = [i for i in range(len(candidate[src])) if legal_remove(candidate[src], i)]
        if not positions:
            return None
        pos = rng.choice(positions)
        node = candidate[src].pop(pos)
        legal = [q for q in range(len(candidate[dst]) + 1) if legal_insert(candidate[dst], node, q)]
        if not legal:
            return None
        # 在合法位置中优先选择飞行距离增量较小的几个，再保留随机性。
        ranked = []
        for q in legal:
            left = 0 if q == 0 else candidate[dst][q - 1]
            right = 0 if q == len(candidate[dst]) else candidate[dst][q]
            inc = (
                data.distance_km[(left, node)]
                + data.distance_km[(node, right)]
                - data.distance_km[(left, right)]
            )
            ranked.append((inc, q))
        ranked.sort()
        q = rng.choice(ranked[: min(4, len(ranked))])[1]
        candidate[dst].insert(q, node)
        changed = (src, dst)
    else:
        i = rng.randrange(len(candidate[src]))
        j = rng.randrange(len(candidate[dst]))
        candidate[src][i], candidate[dst][j] = candidate[dst][j], candidate[src][i]
        if not route_valid(candidate[src]) or not route_valid(candidate[dst]):
            return None
        changed = (src, dst)

    candidate = normalize_changed(candidate, data, changed)
    if not all(route_valid(r) for r in candidate):
        return None
    return candidate


def anneal_seed(
    initial: list[list[int]],
    data: CaseData,
    cap_h: float,
    seed: int,
    iterations: int,
    deadline: float,
) -> tuple[list[list[int]], dict]:
    rng = random.Random(seed)
    current = [two_opt(r, data, max_passes=20) for r in initial]
    _, current_times = all_metrics(current, data)
    current_key = objective(current_times)
    if current_key[1] > cap_h + TOL:
        # 问题一初始路线必须始终可作为保底方案。
        current = [r[:] for r in initial]
        _, current_times = all_metrics(current, data)
        current_key = objective(current_times)
    best = [r[:] for r in current]
    best_key = current_key
    accepted = 0
    attempted = 0

    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        candidate = propose(current, data, rng)
        if candidate is None:
            continue
        attempted += 1
        _, times = all_metrics(candidate, data)
        key = objective(times)
        if key[1] > cap_h + TOL or key[1] > 9.0 + TOL:
            continue

        accept = better(key, current_key)
        if not accept:
            progress = iteration / max(1, iterations - 1)
            temperature = 0.02 * (1.0 - progress) + 0.00015
            loss = max(0.0, key[0] - current_key[0])
            loss += 0.20 * max(0.0, key[1] - current_key[1])
            loss += 0.002 * max(0.0, key[2] - current_key[2])
            accept = rng.random() < math.exp(-loss / temperature)
        if accept:
            current = candidate
            current_key = key
            accepted += 1
            if better(key, best_key):
                best = [r[:] for r in candidate]
                best_key = key

    return best, {
        "seed": seed,
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "best_delta_h": best_key[0],
        "best_Tmax_h": best_key[1],
        "best_total_h": best_key[2],
    }


def deterministic_polish(
    initial: list[list[int]], data: CaseData, cap_h: float, max_rounds: int = 80
) -> list[list[int]]:
    """ 系统枚举重载到轻载的单任务迁移，直到没有词典序改善。"""
    routes = [r[:] for r in initial]
    for _ in range(max_rounds):
        _, times = all_metrics(routes, data)
        old_key = objective(times)
        order = sorted(range(len(routes)), key=times.__getitem__)
        sources = order[-min(2, len(order)) :]
        targets = order[: min(2, len(order))]
        best_candidate = None
        best_key = old_key
        for src in sources:
            for dst in targets:
                if src == dst:
                    continue
                for i, node in enumerate(routes[src]):
                    if not legal_remove(routes[src], i):
                        continue
                    for q in range(len(routes[dst]) + 1):
                        if not legal_insert(routes[dst], node, q):
                            continue
                        candidate = [r[:] for r in routes]
                        candidate[src].pop(i)
                        candidate[dst].insert(q, node)
```

```
                        candidate = normalize_changed(candidate, data, (src, dst))
                        _, cand_times = all_metrics(candidate, data)
                        key = objective(cand_times)
                        if key[1] <= cap_h + TOL and key[1] <= 9.0 + TOL and better(key, best_key):
                            best_candidate = candidate
                            best_key = key
        if best_candidate is None:
            break
        routes = best_candidate
    # 最终把每条路线充分缩短，避免通过保留可消除的绕行来抬高 Tmin。
    return [two_opt(route, data, max_passes=100) for route in routes]


def validate_case(
    case_name: str,
    routes: list[list[int]],
    data: CaseData,
    q1_case: dict,
    epsilon_h: float,
) -> tuple[list[float], list[float]]:
    assert len(routes) == int(q1_case["N"]), f"{case_name}: 无人机数量发生变化"
    assert all(route_valid(r) for r in routes), f"{case_name}: 存在空路线或连续相同巡检点"
    actual = Counter(x for route in routes for x in route)
    assert actual == Counter(data.expected_visits), f"{case_name}: 巡检次数不一致"
    distances, times = all_metrics(routes, data)
    cap = float(q1_case["Tmax"]) + epsilon_h
    assert max(times) <= cap + 1.0e-8, f"{case_name}: 超过问题一完成时间上界"
    assert max(times) <= 9.0 + 1.0e-8, f"{case_name}: 超过 9 小时"
    return distances, times


def solve_case(
    case_name: str,
    data: CaseData,
    q1_case: dict,
    epsilon_h: float,
    seeds: int,
    iterations: int,
    time_limit_s: float,
) -> dict:
    initial = [list(map(int, route["sequence"][1:-1])) for route in q1_case["routes"]]
    validate_case(case_name, initial, data, q1_case, epsilon_h)
    cap = float(q1_case["Tmax"]) + epsilon_h
    deadline = time.monotonic() + time_limit_s
    best = [r[:] for r in initial]
    _, best_times = all_metrics(best, data)
    best_key = objective(best_times)
    history = []

    for offset in range(seeds):
        if time.monotonic() >= deadline:
            break
        routes, record = anneal_seed(
            initial, data, cap, 20260818 + offset, iterations, deadline
        )
        history.append(record)
        _, times = all_metrics(routes, data)
        key = objective(times)
        if better(key, best_key):
            best, best_key = routes, key

    best = deterministic_polish(best, data, cap)
    distances, times = validate_case(case_name, best, data, q1_case, epsilon_h)
    delta1 = float(q1_case["Tmax"]) - float(q1_case["Tmin"])
    delta2 = max(times) - min(times)
    route_records = []
    for k, (route, distance, work_h) in enumerate(zip(best, distances, times), start=1):
        route_records.append(
            {
                "uav": k,
                "sequence": [0] + route + [0],
                "distance_km": distance,
                "service_count": len(route),
                "time_h": work_h,
                "waiting_time_h": 0.0,
            }
        )
    return {
        "N": int(q1_case["N"]),
        "q1_Tmax_h": float(q1_case["Tmax"]),
        "q1_Tmin_h": float(q1_case["Tmin"]),
        "q1_delta_h": delta1,
        "epsilon_h": epsilon_h,
        "completion_time_cap_h": cap,
        "Tmax": max(times),
        "Tmin": min(times),
        "delta": delta2,
        "total_work_h": sum(times),
        "delta_reduction_h": delta1 - delta2,
        "delta_reduction_pct": 0.0 if delta1 <= TOL else (delta1 - delta2) / delta1,
        "objective_order": ["delta", "Tmax", "total_work_h"],
        "waiting_allowed": False,
        "routes": route_records,
        "search_history": history,
        "validation": "PASSED",
    }


def write_csv_outputs(results: dict, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    with (output_dir / "summary.csv").open("w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["Case", "N", "Tmax_h", "Tmin_h", "delta_h", "total_work_h"])
        for case_name, result in results.items():
            w.writerow([case_name, result["N"], result["Tmax"], result["Tmin"], result["delta"], result["total_work_h"]])
    with (output_dir / "comparison.csv").open("w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["Case", "q1_delta_h", "q2_delta_h", "reduction_h", "reduction_pct"])
        for case_name, result in results.items():
            w.writerow([case_name, result["q1_delta_h"], result["delta"], result["delta_reduction_h"], result["delta_reduction_pct"]])
    for case_name, result in results.items():
        max_len = max(len(route["sequence"]) for route in result["routes"])
        headers = ["UAV_ID"] + [f"Stop_{i}" for i in range(1, max_len + 1)] + ["Distance_km", "Service_Count", "Working_Time_h"]
        with (output_dir / f"{case_name}_routes.csv").open("w", newline="", encoding="utf-8-sig") as f:
            w = csv.writer(f)
            w.writerow(headers)
            for route in result["routes"]:
                padding = [""] * (max_len - len(route["sequence"]))
                w.writerow([route["uav"], *route["sequence"], *padding, route["distance_km"], route["service_count"], route["time_h"]])


def validate_saved(input_path: Path, q1_path: Path, q2_path: Path) -> None:
    q1 = json.loads(q1_path.read_text(encoding="utf-8"))
    q2 = json.loads(q2_path.read_text(encoding="utf-8"))
    book = pd.ExcelFile(input_path)
    assert set(q1) == set(q2) == set(book.sheet_names)
    for case_name in book.sheet_names:
        data = load_case(book, case_name)
        routes = [route["sequence"][1:-1] for route in q2[case_name]["routes"]]
        distances, times = validate_case(case_name, routes, data, q1[case_name], q2[case_name]["epsilon_h"])
        assert abs(max(times) - q2[case_name]["Tmax"]) < 1.0e-8
        assert abs(min(times) - q2[case_name]["Tmin"]) < 1.0e-8
        for saved, distance, work_h in zip(q2[case_name]["routes"], distances, times):
```

```
            assert abs(saved["distance_km"] - distance) < 1.0e-8
            assert abs(saved["time_h"] - work_h) < 1.0e-8
            assert saved["waiting_time_h"] == 0.0


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, required=True)
    parser.add_argument("--q1-solution", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--epsilon", type=float, default=0.0)
    parser.add_argument("--seeds", type=int, default=8)
    parser.add_argument("--iterations", type=int, default=30000)
    parser.add_argument("--time-limit-per-case", type=float,
    ↪  default=45.0)
    parser.add_argument("--validate-only", action="store_true")
    args = parser.parse_args()

    output_json = args.output_dir / "q2_solution.json"
    if args.validate_only:
        validate_saved(args.input, args.q1_solution, output_json)
        print(" 问题二保存结果独立校验通过。")
        return

    if args.epsilon < 0:
        raise ValueError("epsilon 必须非负")
    q1 = json.loads(args.q1_solution.read_text(encoding="utf-8"))
    book = pd.ExcelFile(args.input)
    results = {}
    for case_name in book.sheet_names:
        started = time.monotonic()
        data = load_case(book, case_name)
        results[case_name] = solve_case(
            case_name,
            data,
            q1[case_name],
            args.epsilon,
            args.seeds,
            args.iterations,
            args.time_limit_per_case,
        )
        r = results[case_name]
        print(
            f"{case_name}: N={r['N']} Tmax={r['Tmax']:.6f} "
            f"Tmin={r['Tmin']:.6f} delta={r['delta']:.6f} "
            f"elapsed={time.monotonic()-started:.1f}s",
            flush=True,
        )
    args.output_dir.mkdir(parents=True, exist_ok=True)
    output_json.write_text(json.dumps(results, ensure_ascii=False,
    ↪  indent=2), encoding="utf-8")
    write_csv_outputs(results, args.output_dir)
    validate_saved(args.input, args.q1_solution, output_json)
    print(f" 问题二全部校验通过：{output_json}")


if __name__ == "__main__":
    main()
```

附录 D　问题三源程序

以下收入算法 A、算法 B 及两套独立验证程序。算法 B 直接调用算法 A 文件中的数据结构和基础时空冲突函数，因此二者共同构成完整可运行程序。绘图、结果筛选和工作簿格式转换脚本仅放入电子支撑材料。

问题三算法 A 主程序

路径：submission_package/question3/program/q3_solver.py

```python
""" 问题三：动态圆形飞行管制下的多无人机协同巡检优化。

固定问题一/二采用的无人机数量，以问题二路线为初始解；时间以 8:00 为零点、分钟为单位。目标按 (Tmax, delta, total_work) 词典序优化。等待仅在航段或下一巡检服务受管制时发生，不插入用于人为改善均衡性的空等待。
"""

from __future__ import annotations

import argparse
import csv
import json
import math
import random
import time
from collections import Counter
from dataclasses import dataclass
from datetime import datetime, timedelta
from pathlib import Path

import pandas as pd


SPEED_KMH = 55.0
UNIT_KM = 0.1
SERVICE_MIN = 5.0
REQ = {"I": 3, "II": 2, "III": 1}
TOL = 1.0e-9


@dataclass(frozen=True)
class Zone:
    zone_id: str
    center: tuple[float, float]
    radius: float
    start_min: float
    end_min: float

    @property
    def active(self) -> bool:
        return self.end_min > self.start_min + TOL


@dataclass(frozen=True)
class CaseData:
    coordinates: dict[int, tuple[float, float]]
    expected_visits: dict[int, int]
    zones: tuple[Zone, ...]


def clock_to_min(value: object) -> float:
    text = str(value).strip()
    parsed = datetime.strptime(text, "%H:%M")
    return (parsed.hour - 8) * 60 + parsed.minute


def min_to_clock(value: float) -> str:
    stamp = datetime(2000, 1, 1, 8, 0) + timedelta(minutes=value)
    return stamp.strftime("%H:%M:%S")


def load_cases(points_path: Path, zones_path: Path) -> dict[str, CaseData]:
    point_book = pd.ExcelFile(points_path)
    zone_book = pd.ExcelFile(zones_path)
    assert point_book.sheet_names == zone_book.sheet_names
    result: dict[str, CaseData] = {}
    for case_name in point_book.sheet_names:
        points = pd.read_excel(point_book, sheet_name=case_name)
        zones_df = pd.read_excel(zone_book, sheet_name=case_name)
        coordinates = {0: (0.0, 0.0)}
        expected: dict[int, int] = {}
        for _, row in points.iterrows():
            point_id = int(row.Point_ID)
            coordinates[point_id] = (float(row.X_Coordinate), float(row.Y_Coordinate))
            expected[point_id] = REQ[str(row.Inspection_Level)]
        zones = tuple(
            Zone(
                str(row.Zone_ID),
                (float(row.Center_X), float(row.Center_Y)),
                float(row.Radius),
                clock_to_min(row.Start_Time),
                clock_to_min(row.End_Time),
            )
            for _, row in zones_df.iterrows()
        )
        result[case_name] = CaseData(coordinates, expected, zones)
    return result


def point_inside(point: tuple[float, float], zone: Zone) -> bool:
    return math.dist(point, zone.center) <= zone.radius + 1.0e-10


def segment_circle_interval(
    a: tuple[float, float], b: tuple[float, float], zone: Zone
) -> tuple[float, float] | None:
    """ 返回线段位于闭圆内的参数区间，线段参数 lambda 属于 [0,1]。"""
    dx, dy = b[0] - a[0], b[1] - a[1]
    fx, fy = a[0] - zone.center[0], a[1] - zone.center[1]
    aa = dx * dx + dy * dy
    if aa <= TOL:
        return (0.0, 1.0) if point_inside(a, zone) else None
    bb = 2.0 * (fx * dx + fy * dy)
    cc = fx * fx + fy * fy - zone.radius * zone.radius
    disc = bb * bb - 4.0 * aa * cc
    if disc < -1.0e-10:
        return None
    disc = max(0.0, disc)
    root = math.sqrt(disc)
    lo = max(0.0, (-bb - root) / (2.0 * aa))
    hi = min(1.0, (-bb + root) / (2.0 * aa))
    if lo > hi + 1.0e-10:
        return None
    return lo, hi


def overlaps(a0: float, a1: float, b0: float, b1: float) -> bool:
    return max(a0, b0) < min(a1, b1) - 1.0e-9


def departure_blocks(
    a: tuple[float, float],
    b: tuple[float, float],
    travel_min: float,
    service_at_b: bool,
    zones: tuple[Zone, ...],
) -> list[tuple[float, float, str, str]]:
    blocks: list[tuple[float, float, str, str]] = []
    for zone in zones:
        if not zone.active:
            continue
```

```
        interval = segment_circle_interval(a, b, zone)
        if interval is not None:
            lo, hi = interval
            blocks.append(
                (
                    zone.start_min - hi * travel_min,
                    zone.end_min - lo * travel_min,
                    zone.zone_id,
                    "flight",
                )
            )
        if service_at_b and point_inside(b, zone):
            blocks.append(
                (
                    zone.start_min - travel_min - SERVICE_MIN,
                    zone.end_min - travel_min,
                    zone.zone_id,
                    "service",
                )
            )
    return blocks


def unsafe_handoff_blocks(
    current: tuple[float, float],
    current_travel_min: float,
    outgoing: list[tuple[float, float, str, str]],
    zones: tuple[Zone, ...],
) -> list[tuple[float, float, str, str]]:
    """把“到点服务后不能在管制圆内等待”的约束前移到上一航段出发时刻。"""
    holding_zones = [zone for zone in zones if zone.active and point_inside(current, zone)]
    if not holding_zones:
        return []
    offset = current_travel_min + SERVICE_MIN
    result: list[tuple[float, float, str, str]] = []
    for block_start, block_end, cause_id, _ in outgoing:
        for holding in holding_zones:
            # 若完成服务时刻 r 落入出站禁用区间，等待到 block_end；
            # 只要 [r, block_end) 与当前点所在管制区重叠，
            # 就必须在上一点延迟。
            unsafe_end = min(block_end, holding.end_min)
            if block_end > holding.start_min + TOL and block_start < unsafe_end - TOL:
                result.append(
                    (
                        block_start - offset,
                        unsafe_end - offset,
                        f"{holding.zone_id}/{cause_id}",
                        "handoff",
                    )
                )
    return result


def earliest_departure(
    ready: float, blocks: list[tuple[float, float, str, str]]
) -> tuple[float, list[str]]:
    depart = ready
    reasons: set[str] = set()
    for _ in range(len(blocks) + 2):
        covering = [item for item in blocks if item[0] - TOL <= depart < item[1] - TOL]
        if not covering:
            return depart, sorted(reasons)
        reasons.update(f"{zone_id}:{kind}" for _, _, zone_id, kind in covering)
        depart = max(item[1] for item in covering)
    raise RuntimeError(" 禁用出发区间合并未收敛")


def wait_is_safe(
    node: int, point: tuple[float, float], start: float, end: float, zones: tuple[Zone, ...]
) -> bool:
    if node == 0 or end <= start + TOL:
        return True
    for zone in zones:
        if zone.active and point_inside(point, zone) and overlaps(start, end, zone.start_min, zone.end_min):
            return False
    return True


def schedule_route(
    tasks: list[int], data: CaseData, debug: bool = False, release_min: float = 0.0
) -> dict | None:
    if not tasks or any(a == b for a, b in zip(tasks, tasks[1:])):
        return None
    sequence = [0] + tasks + [0]
    leg_data = []
    effective_blocks: list[list[tuple[float, float, str, str]]] = []
    for a_id, b_id in zip(sequence, sequence[1:]):
        a, b = data.coordinates[a_id], data.coordinates[b_id]
        distance = math.dist(a, b) * UNIT_KM
        travel = distance / SPEED_KMH * 60.0
        leg_data.append((a_id, b_id, a, b, distance, travel))
        effective_blocks.append(departure_blocks(a, b, travel, b_id != 0, data.zones))
    # 从路线末端向前传递：若下一航段会迫使无人机在管制圆内等待，
    # 则把相应禁用时段推回到上一安全节点的出发决策。
    for idx in range(len(leg_data) - 2, -1, -1):
        _, b_id, _, b, _, travel = leg_data[idx]
        if b_id != 0:
            effective_blocks[idx].extend(
                unsafe_handoff_blocks(b, travel, effective_blocks[idx + 1], data.zones)
            )

    now = release_min
    distance_km = 0.0
    wait_min = release_min
    events: list[dict] = []
    if release_min > TOL:
        events.append(
            {
                "type": "wait",
                "node": 0,
                "start_min": 0.0,
                "end_min": release_min,
                "duration_min": release_min,
                "reason_zone_ids": ["safe_release_after_all_zones"],
            }
        )
    for leg, (a_id, b_id, a, b, distance, travel) in enumerate(leg_data, start=1):
        blocks = effective_blocks[leg - 1]
        depart, reasons = earliest_departure(now, blocks)
        if not wait_is_safe(a_id, a, now, depart, data.zones):
            if debug:
                print(
                    f"unsafe wait: node={a_id}, ready={now:.3f}, depart={depart:.3f}, "
                    f"next={b_id}, reasons={reasons}",
                    flush=True,
                )
            return None
        if depart > now + TOL:
            events.append(
                {
                    "type": "wait",
                    "node": a_id,
                    "start_min": now,
                    "end_min": depart,
                    "duration_min": depart - now,
                    "reason_zone_ids": reasons,
                }
            )
            wait_min += depart - now
        arrival = depart + travel
        events.append(
            {
                "type": "flight",
                "leg": leg,
                "from": a_id,
                "to": b_id,
                "start_min": depart,
                "end_min": arrival,
                "duration_min": travel,
                "distance_km": distance,
            }
        )
```

```python
        distance_km += distance
        now = arrival
        if b_id != 0:
            events.append(
                {
                    "type": "service",
                    "node": b_id,
                    "start_min": now,
                    "end_min": now + SERVICE_MIN,
                    "duration_min": SERVICE_MIN,
                }
            )
            now += SERVICE_MIN
    return {
        "sequence": sequence,
        "distance_km": distance_km,
        "service_count": len(tasks),
        "flight_time_h": distance_km / SPEED_KMH,
        "wait_time_h": wait_min / 60.0,
        "time_h": now / 60.0,
        "return_clock": min_to_clock(now),
        "events": events,
    }


def valid_tasks(tasks: list[int]) -> bool:
    return bool(tasks) and all(a != b for a, b in zip(tasks, tasks[1:]))


class Evaluator:
    def __init__(self, data: CaseData):
        self.data = data
        self.cache: dict[tuple[int, ...], dict | None] = {}

    def route(self, tasks: list[int]) -> dict | None:
        key = tuple(tasks)
        if key not in self.cache:
            result = schedule_route(tasks, self.data)
            if result is None:
                safe_release = max((zone.end_min for zone in self.data.zones if zone.active), default=0.0)
                result = schedule_route(tasks, self.data, release_min=safe_release)
            self.cache[key] = result
        return self.cache[key]

    def solution(self, routes: list[list[int]]) -> tuple[tuple[float, float, float], list[dict]] | None:
        schedules = [self.route(route) for route in routes]
        if any(item is None for item in schedules):
            return None
        valid = [item for item in schedules if item is not None]
        times = [item["time_h"] for item in valid]
        key = (max(times), max(times) - min(times), sum(times))
        return key, valid


def lex_better(a: tuple[float, ...], b: tuple[float, ...]) -> bool:
    for x, y in zip(a, b):
        if x < y - TOL:
            return True
        if x > y + TOL:
            return False
    return False


def propose(routes: list[list[int]], times: list[float], rng: random.Random) -> list[list[int]] | None:
    candidate = [route[:] for route in routes]
    move = rng.random()
    order = sorted(range(len(routes)), key=times.__getitem__)

    if move < 0.48:
        src = rng.choice(order[-min(2, len(order)):]) if rng.random() < 0.82 else rng.randrange(len(routes))
        choices = [idx for idx in range(len(routes)) if idx != src]
        dst = rng.choice(order[: min(2, len(order))]) if rng.random() < 0.82 else rng.choice(choices)
        if dst == src:
            dst = rng.choice(choices)
        if len(candidate[src]) <= 1:
            return None
        pos = rng.randrange(len(candidate[src]))
        node = candidate[src].pop(pos)
        legal = [
            q for q in range(len(candidate[dst]) + 1)
            if (q == 0 or candidate[dst][q - 1] != node)
            and (q == len(candidate[dst]) or candidate[dst][q] != node)
        ]
        if not legal:
            return None
        candidate[dst].insert(rng.choice(legal), node)
    elif move < 0.73:
        a, b = rng.sample(range(len(routes)), 2)
        ia, ib = rng.randrange(len(candidate[a])), rng.randrange(len(candidate[b]))
        candidate[a][ia], candidate[b][ib] = candidate[b][ib], candidate[a][ia]
    else:
        ridx = rng.randrange(len(routes))
        route = candidate[ridx]
        if len(route) < 4:
            return None
        i, j = sorted(rng.sample(range(len(route)), 2))
        if i == j:
            return None
        route[i : j + 1] = reversed(route[i : j + 1])

    if not all(valid_tasks(route) for route in candidate):
        return None
    return candidate


def search_seed(
    initial: list[list[int]],
    evaluator: Evaluator,
    seed: int,
    iterations: int,
    deadline: float,
) -> tuple[list[list[int]], tuple[float, float, float], dict]:
    rng = random.Random(seed)
    current = [route[:] for route in initial]
    evaluated = evaluator.solution(current)
    if evaluated is None:
        raise RuntimeError("问题二初始路线在问题三调度规则下不可行")
    current_key, current_schedules = evaluated
    best = [route[:] for route in current]
    best_key = current_key
    attempted = accepted = 0

    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        times = [item["time_h"] for item in current_schedules]
        candidate = propose(current, times, rng)
        if candidate is None:
            continue
        attempted += 1
        evaluated = evaluator.solution(candidate)
        if evaluated is None:
            continue
        key, schedules = evaluated
        accept = lex_better(key, current_key)
        if not accept:
            progress = iteration / max(1, iterations - 1)
            temperature = 0.12 * (1.0 - progress) + 0.002
            current_score = current_key[0] + 0.18 * current_key[1] + 0.002 * current_key[2]
            candidate_score = key[0] + 0.18 * key[1] + 0.002 * key[2]
            loss = max(0.0, candidate_score - current_score)
            accept = rng.random() < math.exp(-loss / temperature)
        if accept:
            current, current_key, current_schedules = candidate, key, schedules
            accepted += 1
            if lex_better(key, best_key):
                best, best_key = [route[:] for route in candidate], key

    return best, best_key, {
        "seed": seed,
```

```
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "Tmax_h": best_key[0],
        "delta_h": best_key[1],
        "total_work_h": best_key[2],
    }


def validate_counts(routes: list[list[int]], data: CaseData) -> None:
    found = Counter(node for route in routes for node in route)
    assert dict(found) == data.expected_visits
    assert all(valid_tasks(route) for route in routes)


def zone_summary(data: CaseData) -> dict:
    active = [zone for zone in data.zones if zone.active]
    involved = {
        node
        for node, point in data.coordinates.items()
        if node != 0 and any(point_inside(point, zone) for zone in active)
    }
    return {
        "zone_count": len(data.zones),
        "positive_duration_count": len(active),
        "zero_duration_count": len(data.zones) - len(active),
        "involved_point_count": len(involved),
        "involved_points": sorted(involved),
    }


def solve_case(
    case_name: str,
    data: CaseData,
    initial: list[list[int]],
    seeds: int,
    iterations: int,
    seconds: float,
) -> dict:
    validate_counts(initial, data)
    evaluator = Evaluator(data)
    baseline = evaluator.solution(initial)
    if baseline is None:
        for idx, route in enumerate(initial, start=1):
            if evaluator.route(route) is None:
                print(f"{case_name} UAV{idx} diagnostic", flush=True)
                schedule_route(route, data, debug=True)
        raise RuntimeError(f"{case_name}: 初始路线无法生成合法动态调度")
    best_routes = [route[:] for route in initial]
    best_key, _ = baseline
    history = []
    deadline = time.monotonic() + seconds
    for offset in range(seeds):
        routes, key, report = search_seed(
            initial,
            evaluator,
            20260818 + offset,
            iterations,
            deadline,
        )
        history.append(report)
        if lex_better(key, best_key):
            best_routes, best_key = routes, key
        if time.monotonic() >= deadline:
            break

    validate_counts(best_routes, data)
    final = evaluator.solution(best_routes)
    assert final is not None
    key, schedules = final
    routes_output = []
    for uav, schedule in enumerate(schedules, start=1):
        routes_output.append({"uav": uav, **schedule})
    return {
        "N": len(best_routes),
        "Tmax": key[0],
        "Tmin": min(item["time_h"] for item in schedules),
        "delta": key[1],
        "total_work_h": key[2],
        "total_wait_min": sum(item["wait_time_h"] * 60 for item in schedules),
        "latest_return": min_to_clock(key[0] * 60),
        "time_origin": "08:00",
        "deadline_17_00": False,
        "objective_order": ["Tmax", "delta", "total_work_h"],
        "zone_summary": zone_summary(data),
        "routes": routes_output,
        "search_history": history,
    }


def write_csv(results: dict, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    with (output_dir / "summary.csv").open("w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.writer(handle)
        writer.writerow(["Case", "N", "Tmax_h", "Tmin_h", "delta_h", "wait_min", "latest_return"])
        for case_name, case in results.items():
            writer.writerow([case_name, case["N"], case["Tmax"], case["Tmin"], case["delta"], case["total_wait_min"], case["latest_return"]])
    for case_name, case in results.items():
        with (output_dir / f"{case_name}_routes.csv").open("w", newline="", encoding="utf-8-sig") as handle:
            writer = csv.writer(handle)
            writer.writerow(["UAV", "Sequence", "Distance_km", "Service_Count", "Wait_min", "Work_h", "Return"])
            for route in case["routes"]:
                writer.writerow([route["uav"], "-".join(map(str, route["sequence"])), route["distance_km"], route["service_count"], route["wait_time_h"] * 60, route["time_h"], route["return_clock"]])


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--q2-solution", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--seeds", type=int, default=5)
    parser.add_argument("--iterations", type=int, default=25000)
    parser.add_argument("--seconds-per-case", type=float, default=35.0)
    args = parser.parse_args()

    cases = load_cases(args.points, args.zones)
    q2 = json.loads(args.q2_solution.read_text(encoding="utf-8"))
    assert set(cases) == set(q2)
    results = {}
    for case_name, data in cases.items():
        initial = [[int(x) for x in route["sequence"][1:-1]] for route in q2[case_name]["routes"]]
        started = time.monotonic()
        results[case_name] = solve_case(
            case_name,
            data,
            initial,
            args.seeds,
            args.iterations,
            args.seconds_per_case,
        )
        elapsed = time.monotonic() - started
        case = results[case_name]
        print(
            f"{case_name}: N={case['N']}, Tmax={case['Tmax']:.6f} h, "
            f"Tmin={case['Tmin']:.6f} h, delta={case['delta']:.6f} h, "
            f"wait={case['total_wait_min']:.3f} min, elapsed={elapsed:.1f}s",
            flush=True,
        )

    args.output_dir.mkdir(parents=True, exist_ok=True)
    output = args.output_dir / "q3_solution.json"
    output.write_text(json.dumps(results, ensure_ascii=False, indent=2), encoding="utf-8")
    write_csv(results, args.output_dir)
    print(output)
```

```
578
579 if __name__ == "__main__":
580     main()
```

问题三算法 B 主程序

路径： submission_package/question3/program/q3_solver_enhanced.py

```
1 """ 问题三算法 B：时变圆形禁飞区下的分层增强求解器。
2
3 与基准算法的区别：
4 1. 每次巡检使用唯一任务副本编号；
5 2. 对受阻转场比较“等待后直飞”和“边界离散可见图绕飞”；
6 3. 阶段 A 只搜索最小 Tmax，阶段 B 在给定 epsilon 容差内搜索最小 delta；
7 4. 固定问题二机队规模是正式方案，额外无人机仅作为敏感性分析。
8
9 圆边界用外扩采样环及直线弦近似。采样环半径除以 cos(pi/m)，保证相邻弦
10 不进入原禁飞圆内部；所有最终飞行事件仍由解析线段—圆—时间区间复核。
11 """
12
13 from __future__ import annotations
14
15 import argparse
16 import csv
17 import heapq
18 import json
19 import math
20 import random
21 import time
22 from collections import Counter, defaultdict
23 from dataclasses import dataclass
24 from pathlib import Path
25
26 import q3_solver as baseline
27
28
29 TOL = 1.0e-9
30
31
32 @dataclass(frozen=True)
33 class GeoNode:
34     label: str
35     xy: tuple[float, float]
36
37
38 def task_point(task_id: str) -> int:
39     return int(task_id.split("#", 1)[0][1:])
40
41
42 def build_unique_copies(initial_physical: list[list[int]]) ->
   ↪ tuple[list[list[str]], dict[str, int]]:
43     counters: defaultdict[int, int] = defaultdict(int)
44     mapping: dict[str, int] = {}
45     routes: list[list[str]] = []
46     for route in initial_physical:
47         copies: list[str] = []
48         for point in route:
49             counters[point] += 1
50             task_id = f"P{point:03d}#{counters[point]}"
51             mapping[task_id] = point
52             copies.append(task_id)
53         routes.append(copies)
54     return routes, mapping
55
56
57 def expected_copy_ids(data: baseline.CaseData) -> set[str]:
58     return {
59         f"P{point:03d}#{copy_no}"
60         for point, count in data.expected_visits.items()
61         for copy_no in range(1, count + 1)
62     }
63
64
65 def valid_copy_route(route: list[str], mapping: dict[str, int]) ->
   ↪ bool:
66     if not route:
67         return False
68     physical = [mapping[item] for item in route]
69     return all(a != b for a, b in zip(physical, physical[1:]))
70
71
72 def validate_copy_routes(
73     routes: list[list[str]], data: baseline.CaseData, mapping:
   ↪ dict[str, int]
74 ) -> None:
75     flat = [item for route in routes for item in route]
76     assert len(flat) == len(set(flat)), " 任务副本编号重复"
77     assert set(flat) == expected_copy_ids(data), " 任务副本缺失或多余"
78     assert set(mapping) == expected_copy_ids(data)
79     assert all(valid_copy_route(route, mapping) for route in routes)
80
81
82 def point_segment_distance(
83     p: tuple[float, float], a: tuple[float, float], b: tuple[float,
   ↪ float]
84 ) -> float:
85     dx, dy = b[0] - a[0], b[1] - a[1]
86     denom = dx * dx + dy * dy
87     if denom <= TOL:
88         return math.dist(p, a)
89     lam = ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy) / denom
90     lam = min(1.0, max(0.0, lam))
91     q = (a[0] + lam * dx, a[1] + lam * dy)
92     return math.dist(p, q)
93
94
95 class DetourRouter:
96     def __init__(
97         self,
98         data: baseline.CaseData,
99         ring_samples: int = 16,
100         safety_margin: float = 0.0,
101     ) -> None:
102         if ring_samples < 8:
103             raise ValueError("ring_samples 至少为 8")
104         self.data = data
105         self.ring_samples = ring_samples
106         self.safety_margin = safety_margin
107         self.path_cache: dict[tuple[int, int, tuple[int, ...]],
    ↪ list[GeoNode] | None] = {}
108
109     def _zone_radius(self, zone: baseline.Zone) -> float:
110         return zone.radius + self.safety_margin
111
112     def _static_clear(
113         self,
114         a: tuple[float, float],
115         b: tuple[float, float],
116         obstacle_indices: tuple[int, ...],
117     ) -> bool:
118         for index in obstacle_indices:
119             zone = self.data.zones[index]
120             radius = self._zone_radius(zone)
121             if point_segment_distance(zone.center, a, b) < radius -
    ↪ 1.0e-8:
122                 return False
123         return True
124
125     def _ring_nodes(self, obstacle_indices: tuple[int, ...]) ->
    ↪ list[GeoNode]:
126         nodes: list[GeoNode] = []
127         inflation = 1.0 / math.cos(math.pi / self.ring_samples)
128         for index in obstacle_indices:
129             zone = self.data.zones[index]
130             radius = self._zone_radius(zone) * inflation + 1.0e-6
131             for sample in range(self.ring_samples):
132                 angle = 2.0 * math.pi * sample / self.ring_samples
133                 xy = (
134                     zone.center[0] + radius * math.cos(angle),
135                     zone.center[1] + radius * math.sin(angle),
136                 )
137                 # 位于其他同时考虑的圆内部的采样点不能作为安全转向点。
138                 if any(
139                     other != index
140                     and math.dist(xy, self.data.zones[other].center)
141                     < self._zone_radius(self.data.zones[other]) -
    ↪ 1.0e-8
```

```
                    for other in obstacle_indices
                ):
                    continue
                nodes.append(GeoNode(f"Z{index + 1}:B{sample}", xy))
        return nodes

    def _static_detour(
        self,
        a_id: int,
        b_id: int,
        obstacles: tuple[int, ...],
    ) -> list[GeoNode] | None:
        key = (a_id, b_id, obstacles)
        if key in self.path_cache:
            return self.path_cache[key]
        source = GeoNode(f"P{a_id}", self.data.coordinates[a_id])
        target = GeoNode(f"P{b_id}", self.data.coordinates[b_id])
        nodes = [source, target, *self._ring_nodes(obstacles)]
        if any(
            math.dist(source.xy, self.data.zones[index].center)
            < self._zone_radius(self.data.zones[index]) - 1.0e-8
            or math.dist(target.xy, self.data.zones[index].center)
            < self._zone_radius(self.data.zones[index]) - 1.0e-8
            for index in obstacles
        ):
            self.path_cache[key] = None
            return None

        adjacency: list[list[tuple[int, float]]] = [[] for _ in
        ↪  nodes]
        for i in range(len(nodes)):
            for j in range(i + 1, len(nodes)):
                if self._static_clear(nodes[i].xy, nodes[j].xy,
                ↪  obstacles):
                    distance = math.dist(nodes[i].xy, nodes[j].xy)
                    adjacency[i].append((j, distance))
                    adjacency[j].append((i, distance))

        dist = [math.inf] * len(nodes)
        previous = [-1] * len(nodes)
        dist[0] = 0.0
        heap: list[tuple[float, int]] = [(0.0, 0)]
        while heap:
            current, u = heapq.heappop(heap)
            if current > dist[u] + TOL:
                continue
            if u == 1:
                break
            for v, weight in adjacency[u]:
                candidate = current + weight
                if candidate < dist[v] - TOL:
                    dist[v] = candidate
                    previous[v] = u
                    heapq.heappush(heap, (candidate, v))
        if not math.isfinite(dist[1]):
            self.path_cache[key] = None
            return None
        order: list[int] = []
        cursor = 1
        while cursor >= 0:
            order.append(cursor)
            if cursor == 0:
                break
            cursor = previous[cursor]
        path = [nodes[index] for index in reversed(order)]
        self.path_cache[key] = path
        return path

    def _simulate_polyline(
        self,
        nodes: list[GeoNode],
        ready: float,
        service_at_target: bool,
        source_is_base: bool,
        mode: str,
    ) -> dict | None:
        now = ready
        total_distance = 0.0
        events: list[dict] = []
        for index, (left, right) in enumerate(zip(nodes, nodes[1:])):
            distance_km = math.dist(left.xy, right.xy) *
            ↪  baseline.UNIT_KM
            travel_min = distance_km / baseline.SPEED_KMH * 60.0
            last = index == len(nodes) - 2
            blocks = baseline.departure_blocks(
                left.xy,
                right.xy,
                travel_min,
                service_at_target and last,
                self.data.zones,
            )
            depart, reasons = baseline.earliest_departure(now,
            ↪  blocks)
            allow_base_wait = source_is_base and index == 0
            node_number = 0 if allow_base_wait else -1
            if not baseline.wait_is_safe(node_number, left.xy, now,
            ↪  depart, self.data.zones):
                return None
            if depart > now + TOL:
                events.append(
                    {
                        "type": "wait",
                        "node_label": left.label,
                        "node_xy": list(left.xy),
                        "start_min": now,
                        "end_min": depart,
                        "duration_min": depart - now,
                        "reason_zone_ids": reasons,
                        "mode": mode,
                    }
                )
            arrival = depart + travel_min
            events.append(
                {
                    "type": "flight",
                    "from_label": left.label,
                    "to_label": right.label,
                    "from_xy": list(left.xy),
                    "to_xy": list(right.xy),
                    "start_min": depart,
                    "end_min": arrival,
                    "duration_min": travel_min,
                    "distance_km": distance_km,
                    "mode": mode,
                }
            )
            total_distance += distance_km
            now = arrival
        return {
            "arrival_min": now,
            "distance_km": total_distance,
            "events": events,
            "mode": mode,
        }

    def travel(
        self,
        a_id: int,
        b_id: int,
        ready: float,
        service_at_target: bool,
    ) -> dict | None:
        direct_nodes = [
            GeoNode(f"P{a_id}", self.data.coordinates[a_id]),
            GeoNode(f"P{b_id}", self.data.coordinates[b_id]),
        ]
        direct = self._simulate_polyline(
            direct_nodes,
            ready,
            service_at_target,
            a_id == 0,
            "direct_or_wait",
        )
        if direct is not None and not any(event["type"] == "wait" for
        ↪  event in direct["events"]):
            return direct

        horizon = direct["arrival_min"] if direct is not None else
        ↪  max(
            (zone.end_min for zone in self.data.zones if
            ↪  zone.active), default=ready
        )
        obstacles = tuple(
```

```
            index
            for index, zone in enumerate(self.data.zones)
            if zone.active
            and baseline.overlaps(ready, max(ready + TOL, horizon), zone.start_min, zone.end_min)
        )
        detour = None
        if obstacles:
            path = self._static_detour(a_id, b_id, obstacles)
            if path is not None and len(path) > 2:
                detour = self._simulate_polyline(
                    path,
                    ready,
                    service_at_target,
                    a_id == 0,
                    "boundary_visibility_detour",
                )
        candidates = [item for item in (direct, detour) if item is not None]
        if not candidates:
            return None
        return min(candidates, key=lambda item: (item["arrival_min"], item["distance_km"]))


def _schedule_copy_route_with_detours(
    route: list[str],
    mapping: dict[str, int],
    data: baseline.CaseData,
    router: DetourRouter,
) -> dict | None:
    if not valid_copy_route(route, mapping):
        return None
    now = 0.0
    distance_km = 0.0
    events: list[dict] = []
    physical = [mapping[item] for item in route]
    point_sequence = [0, *physical, 0]
    task_sequence = ["BASE", *route, "BASE"]
    for leg, (a_id, b_id) in enumerate(zip(point_sequence, point_sequence[1:]), start=1):
        movement = router.travel(a_id, b_id, now, b_id != 0)
        if movement is None:
            return None
        for event in movement["events"]:
            event = {"leg": leg, **event}
            events.append(event)
        now = movement["arrival_min"]
        distance_km += movement["distance_km"]
        if b_id != 0:
            task_id = route[leg - 1]
            events.append(
                {
                    "type": "service",
                    "task_id": task_id,
                    "point_id": b_id,
                    "node_xy": list(data.coordinates[b_id]),
                    "start_min": now,
                    "end_min": now + baseline.SERVICE_MIN,
                    "duration_min": baseline.SERVICE_MIN,
                }
            )
            now += baseline.SERVICE_MIN
    wait_min = sum(event["duration_min"] for event in events if event["type"] == "wait")
    detour_legs = len(
        {
            event["leg"]
            for event in events
            if event["type"] == "flight" and event.get("mode") == "boundary_visibility_detour"
        }
    )
    return {
        "sequence": point_sequence,
        "task_sequence": task_sequence,
        "distance_km": distance_km,
        "service_count": len(route),
        "flight_time_h": distance_km / baseline.SPEED_KMH,
        "wait_time_h": wait_min / 60.0,
        "time_h": now / 60.0,
        "return_clock": baseline.min_to_clock(now),
        "detour_leg_count": detour_legs,
        "used_baseline_handoff_fallback": False,
        "events": events,
    }


def _baseline_handoff_fallback(
    route: list[str], mapping: dict[str, int], data: baseline.CaseData
) -> dict | None:
    """ 当逐航段绕飞评价陷入不安全驻留时，使用基准算法的反向等待传播。"""
    physical = [mapping[item] for item in route]
    scheduled = baseline.schedule_route(physical, data)
    if scheduled is None:
        safe_release = max((zone.end_min for zone in data.zones if zone.active), default=0.0)
        scheduled = baseline.schedule_route(physical, data, release_min=safe_release)
    if scheduled is None:
        return None
    service_cursor = 0
    converted: list[dict] = []
    for event in scheduled["events"]:
        item = dict(event)
        if event["type"] == "flight":
            a_id, b_id = int(event["from"]), int(event["to"])
            item.update(
                {
                    "from_label": f"P{a_id}",
                    "to_label": f"P{b_id}",
                    "from_xy": list(data.coordinates[a_id]),
                    "to_xy": list(data.coordinates[b_id]),
                    "mode": "direct_with_backward_handoff",
                }
            )
        elif event["type"] == "wait":
            node = int(event["node"])
            item.update(
                {
                    "node_label": f"P{node}",
                    "node_xy": list(data.coordinates[node]),
                    "mode": "direct_with_backward_handoff",
                }
            )
        elif event["type"] == "service":
            item.update(
                {
                    "task_id": route[service_cursor],
                    "point_id": int(event["node"]),
                    "node_xy": list(data.coordinates[int(event["node"])]),
                }
            )
            service_cursor += 1
        converted.append(item)
    return {
        **{key: value for key, value in scheduled.items() if key != "events"},
        "task_sequence": ["BASE", *route, "BASE"],
        "detour_leg_count": 0,
        "used_baseline_handoff_fallback": True,
        "events": converted,
    }


def schedule_copy_route(
    route: list[str],
    mapping: dict[str, int],
    data: baseline.CaseData,
    router: DetourRouter,
) -> dict | None:
    enhanced = _schedule_copy_route_with_detours(route, mapping, data, router)
    if enhanced is not None:
        return enhanced
    return _baseline_handoff_fallback(route, mapping, data)


class EnhancedEvaluator:
    def __init__(
        self,
```

```
        data: baseline.CaseData,
        mapping: dict[str, int],
        ring_samples: int,
        safety_margin: float,
    ) -> None:
        self.data = data
        self.mapping = mapping
        self.router = DetourRouter(data, ring_samples, safety_margin)
        self.cache: dict[tuple[str, ...], dict | None] = {}

    def route(self, tasks: list[str]) -> dict | None:
        key = tuple(tasks)
        if key not in self.cache:
            self.cache[key] = schedule_copy_route(tasks, self.mapping, self.data, self.router)
        return self.cache[key]

    def solution(self, routes: list[list[str]]) -> tuple[tuple[float, float, float], list[dict]] | None:
        schedules = [self.route(route) for route in routes]
        if any(item is None for item in schedules):
            return None
        valid = [item for item in schedules if item is not None]
        times = [item["time_h"] for item in valid]
        return (max(times), max(times) - min(times), sum(times)), valid


def propose(
    routes: list[list[str]],
    times: list[float],
    mapping: dict[str, int],
    rng: random.Random,
) -> list[list[str]] | None:
    candidate = [route[:] for route in routes]
    order = sorted(range(len(routes)), key=times.__getitem__)
    move = rng.random()
    if move < 0.42:
        src = rng.choice(order[-min(2, len(order)):]) if rng.random() < 0.8 else rng.randrange(len(routes))
        choices = [idx for idx in range(len(routes)) if idx != src]
        dst = rng.choice(order[: min(2, len(order))]) if rng.random() < 0.8 else rng.choice(choices)
        if len(candidate[src]) <= 1:
            return None
        task = candidate[src].pop(rng.randrange(len(candidate[src])))
        point = mapping[task]
        legal = [
            pos
            for pos in range(len(candidate[dst]) + 1)
            if (pos == 0 or mapping[candidate[dst][pos - 1]] != point)
            and (pos == len(candidate[dst]) or mapping[candidate[dst][pos]] != point)
        ]
        if not legal:
            return None
        candidate[dst].insert(rng.choice(legal), task)
    elif move < 0.68:
        a, b = rng.sample(range(len(routes)), 2)
        ia, ib = rng.randrange(len(candidate[a])), rng.randrange(len(candidate[b]))
        candidate[a][ia], candidate[b][ib] = candidate[b][ib], candidate[a][ia]
    elif move < 0.84:
        ridx = rng.randrange(len(routes))
        if len(candidate[ridx]) < 4:
            return None
        i, j = sorted(rng.sample(range(len(candidate[ridx])), 2))
        candidate[ridx][i : j + 1] = reversed(candidate[ridx][i : j + 1])
    else:
        a, b = rng.sample(range(len(routes)), 2)
        if len(candidate[a]) < 2 or len(candidate[b]) < 2:
            return None
        ia, ja = sorted(rng.sample(range(len(candidate[a]) + 1), 2))
        ib, jb = sorted(rng.sample(range(len(candidate[b]) + 1), 2))
        if ia == ja or ib == jb:
            return None
        seg_a, seg_b = candidate[a][ia:ja], candidate[b][ib:jb]
        candidate[a][ia:ja], candidate[b][ib:jb] = seg_b, seg_a
    if not all(valid_copy_route(route, mapping) for route in candidate):
        return None
    return candidate


def stage_a_search(
    initial: list[list[str]],
    evaluator: EnhancedEvaluator,
    mapping: dict[str, int],
    rng: random.Random,
    iterations: int,
    deadline: float,
) -> tuple[list[list[str]], tuple[float, float, float], dict]:
    current = [route[:] for route in initial]
    evaluated = evaluator.solution(current)
    if evaluated is None:
        raise RuntimeError("增强算法无法调度初始路线")
    current_key, current_schedules = evaluated
    best, best_key = [route[:] for route in current], current_key
    attempted = accepted = 0
    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        candidate = propose(current, [item["time_h"] for item in current_schedules], mapping, rng)
        if candidate is None:
            continue
        attempted += 1
        evaluated = evaluator.solution(candidate)
        if evaluated is None:
            continue
        key, schedules = evaluated
        progress = iteration / max(1, iterations - 1)
        temperature = 0.10 * (1.0 - progress) + 0.001
        loss = key[0] - current_key[0]
        accept = loss < -TOL or rng.random() < math.exp(-max(0.0, loss) / temperature)
        if accept:
            current, current_key, current_schedules = candidate, key, schedules
            accepted += 1
            if key[0] < best_key[0] - TOL or (
                abs(key[0] - best_key[0]) <= TOL and key[2] < best_key[2] - TOL
            ):
                best, best_key = [route[:] for route in candidate], key
    return best, best_key, {
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "Tmax_star_h": best_key[0],
        "secondary_metrics_not_optimized": {"delta_h": best_key[1], "total_work_h": best_key[2]},
    }


def stage_b_search(
    stage_a_routes: list[list[str]],
    tmax_star: float,
    epsilon: float,
    evaluator: EnhancedEvaluator,
    mapping: dict[str, int],
    rng: random.Random,
    iterations: int,
    deadline: float,
) -> tuple[list[list[str]], tuple[float, float, float], dict]:
    threshold = (1.0 + epsilon) * tmax_star
    current = [route[:] for route in stage_a_routes]
    evaluated = evaluator.solution(current)
    assert evaluated is not None
    current_key, current_schedules = evaluated
    best, best_key = [route[:] for route in current], current_key
    attempted = accepted = rejected_by_threshold = 0
    for iteration in range(iterations):
        if time.monotonic() >= deadline:
            break
        candidate = propose(current, [item["time_h"] for item in current_schedules], mapping, rng)
        if candidate is None:
            continue
        attempted += 1
```

```
        evaluated = evaluator.solution(candidate)
        if evaluated is None:
            continue
        key, schedules = evaluated
        if key[0] > threshold + TOL:
            rejected_by_threshold += 1
            continue
        current_obj = (current_key[1], current_key[2],
        ↪ current_key[0])
        candidate_obj = (key[1], key[2], key[0])
        progress = iteration / max(1, iterations - 1)
        temperature = 0.06 * (1.0 - progress) + 0.0005
        loss = (candidate_obj[0] - current_obj[0]) + 0.002 * (
            candidate_obj[1] - current_obj[1]
        )
        accept = candidate_obj < current_obj or rng.random() <
        ↪ math.exp(-max(0.0, loss) / temperature)
        if accept:
            current, current_key, current_schedules = candidate, key,
            ↪ schedules
            accepted += 1
            if (key[1], key[2], key[0]) < (best_key[1], best_key[2],
            ↪ best_key[0]):
                best, best_key = [route[:] for route in candidate],
                ↪ key
    return best, best_key, {
        "epsilon": epsilon,
        "Tmax_limit_h": threshold,
        "attempted_moves": attempted,
        "accepted_moves": accepted,
        "rejected_by_Tmax_limit": rejected_by_threshold,
        "delta_h": best_key[1],
        "Tmax_h": best_key[0],
        "total_work_h": best_key[2],
    }


def add_uavs(initial: list[list[str]], extra: int) ->
↪ list[list[str]]:
    routes = [route[:] for route in initial]
    for _ in range(extra):
        src = max(range(len(routes)), key=lambda idx:
        ↪ len(routes[idx]))
        if len(routes[src]) <= 1:
            raise ValueError(" 无法继续拆分非空路线")
        midpoint = len(routes[src]) // 2
        new_route = routes[src][midpoint:]
        routes[src] = routes[src][:midpoint]
        routes.append(new_route)
    return routes


def solve_case_n(
    case_name: str,
    data: baseline.CaseData,
    initial: list[list[str]],
    mapping: dict[str, int],
    epsilon: float,
    ring_samples: int,
    safety_margin: float,
    stage_a_iterations: int,
    stage_b_iterations: int,
    seconds: float,
    seed: int,
) -> dict:
    validate_copy_routes(initial, data, mapping)
    evaluator = EnhancedEvaluator(data, mapping, ring_samples,
    ↪ safety_margin)
    started = time.monotonic()
    phase_deadline = started + seconds / 2.0
    rng = random.Random(seed)
    stage_a_routes, stage_a_key, stage_a_report = stage_a_search(
        initial, evaluator, mapping, rng, stage_a_iterations,
        ↪ phase_deadline
    )
    original_stage_a_tmax = stage_a_key[0]
    stage_b_routes, stage_b_key, stage_b_report = stage_b_search(
        stage_a_routes,
        stage_a_key[0],
        epsilon,
        evaluator,
        mapping,
        rng,
        stage_b_iterations,
        started + seconds,
    )
    refinements: list[dict] = []
    # 启发式阶段 B 可能在探索均衡解时发现更小的 Tmax。此时必须将其回代为
    # 新的阶段A当前最好值，并按收紧后的阈值重新执行阶段B，避免目标错位。
    for refinement in range(3):
        if stage_b_key[0] >= stage_a_key[0] - TOL:
            break
        refinements.append(
            {
                "round": refinement + 1,
                "old_Tmax_star_h": stage_a_key[0],
                "new_Tmax_star_h": stage_b_key[0],
            }
        )
        stage_a_routes = [route[:] for route in stage_b_routes]
        stage_a_key = stage_b_key
        stage_b_routes, stage_b_key, stage_b_report = stage_b_search(
            stage_a_routes,
            stage_a_key[0],
            epsilon,
            evaluator,
            mapping,
            rng,
            stage_b_iterations,
            time.monotonic() + max(2.0, seconds / 3.0),
        )
    stage_a_report["initial_Tmax_star_h"] = original_stage_a_tmax
    stage_a_report["Tmax_star_h"] = stage_a_key[0]
    stage_a_report["refined_from_stage_b"] = bool(refinements)
    stage_a_report["refinement_history"] = refinements
    validate_copy_routes(stage_b_routes, data, mapping)
    evaluated = evaluator.solution(stage_b_routes)
    assert evaluated is not None
    key, schedules = evaluated
    assert key[0] <= (1.0 + epsilon) * stage_a_key[0] + TOL
    return {
        "case": case_name,
        "N": len(stage_b_routes),
        "Tmax": key[0],
        "Tmin": min(item["time_h"] for item in schedules),
        "delta": key[1],
        "total_work_h": key[2],
        "total_wait_min": sum(item["wait_time_h"] * 60.0 for item in
        ↪ schedules),
        "latest_return": baseline.min_to_clock(key[0] * 60.0),
        "time_origin": "08:00",
        "deadline_17_00": False,
        "zone_policy": "half_open; zero-duration zones excluded",
        "task_copy_policy": "unique task IDs Pxxx#copy_no",
        "detour_model": {
            "type": "sampled boundary visibility graph",
            "ring_samples": ring_samples,
            "safety_margin_coordinate_units": safety_margin,
        },
        "stage_a": stage_a_report,
        "stage_b": stage_b_report,
        "routes": [
            {"uav": uav, **schedule}
            for uav, schedule in enumerate(schedules, start=1)
        ],
    }


def write_summary(results: dict, output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    with (output_dir / "summary_enhanced.csv").open("w", newline="",
    ↪ encoding="utf-8-sig") as handle:
        writer = csv.writer(handle)
        writer.writerow(
            [
                "Case",
                "Scenario",
                "N",
                "StageA_TmaxStar_h",
                "Epsilon",
                "Final_Tmax_h",
                "Final_Tmin_h",
                "Final_delta_h",
```

```
                "Wait_min",
                "Detour_legs",
                "Latest_return",
            ]
        )
        for case_name, scenarios in results.items():
            for scenario_name, item in scenarios.items():
                writer.writerow(
                    [
                        case_name,
                        scenario_name,
                        item["N"],
                        item["stage_a"]["Tmax_star_h"],
                        item["stage_b"]["epsilon"],
                        item["Tmax"],
                        item["Tmin"],
                        item["delta"],
                        item["total_wait_min"],
                        sum(route["detour_leg_count"] for route in item["routes"]),
                        item["latest_return"],
                    ]
                )


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--q2-solution", type=Path, required=True)
    parser.add_argument(
        "--baseline-q3-solution",
        type=Path,
        help="可选：算法A的 q3_solution.json；提供后作为算法B的必选初始种子",
    )
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--epsilon", type=float, default=0.005)
    parser.add_argument("--ring-samples", type=int, default=16)
    parser.add_argument("--safety-margin", type=float, default=0.0)
    parser.add_argument("--stage-a-iterations", type=int, default=800)
    parser.add_argument("--stage-b-iterations", type=int, default=800)
    parser.add_argument("--seconds-per-case", type=float, default=45.0)
    parser.add_argument("--extra-uavs", type=int, default=0)
    args = parser.parse_args()
    if not 0.0 <= args.epsilon <= 0.05:
        raise ValueError("epsilon 建议位于 [0,0.05]")
    if args.extra_uavs < 0:
        raise ValueError("extra-uavs 不能为负")

    cases = baseline.load_cases(args.points, args.zones)
    q2 = json.loads(args.q2_solution.read_text(encoding="utf-8"))
    assert set(cases) == set(q2)
    baseline_q3 = None
    if args.baseline_q3_solution is not None:
        baseline_q3 = json.loads(args.baseline_q3_solution.read_text(encoding="utf-8"))
        assert set(cases) == set(baseline_q3)
    results: dict[str, dict] = {}
    for case_index, (case_name, data) in enumerate(cases.items()):
        seed_source = baseline_q3[case_name] if baseline_q3 is not None else q2[case_name]
        physical = [
            [int(value) for value in route["sequence"][1:-1]]
            for route in seed_source["routes"]
        ]
        initial, mapping = build_unique_copies(physical)
        expected = expected_copy_ids(data)
        if set(mapping) != expected:
            raise RuntimeError(f"{case_name}: 问题二路线与任务副本需求不一致")
        scenarios: dict[str, dict] = {}
        scenario_routes = [route[:] for route in initial]
        for extra in range(args.extra_uavs + 1):
            scenario_name = "formal_fixed_N0" if extra == 0 else f"sensitivity_N0_plus_{extra}"
            result = solve_case_n(
                case_name,
                data,
                scenario_routes,
                mapping,
                args.epsilon,
                args.ring_samples,
                args.safety_margin,
                args.stage_a_iterations,
                args.stage_b_iterations,
                args.seconds_per_case,
                20260818 + 1000 * case_index + extra,
            )
            scenarios[scenario_name] = result
            print(
                f"{case_name} {scenario_name}: N={result['N']} "
                f"Tmax*={result['stage_a']['Tmax_star_h']:.6f} "
                f"Tmax={result['Tmax']:.6f} delta={result['delta']:.6f} "
                f"wait={result['total_wait_min']:.3f}",
                flush=True,
            )
            # 机队规模敏感性采用逐级热启动：下一规模从当前已优化方案拆出一条
            # 新路线，而不是每次回到未经优化的原始路线重新硬拆。
            if extra < args.extra_uavs:
                optimized_routes = [route["task_sequence"][1:-1] for route in result["routes"]]
                scenario_routes = add_uavs(optimized_routes, 1)
        results[case_name] = scenarios

    args.output_dir.mkdir(parents=True, exist_ok=True)
    output = args.output_dir / "q3_enhanced_solution.json"
    output.write_text(json.dumps(results, ensure_ascii=False, indent=2), encoding="utf-8")
    write_summary(results, args.output_dir)
    print(output)


if __name__ == "__main__":
    main()
```

问题三快速独立校验程序

路径： submission_package/question3/program/q3_fast_validator.py

```
""" 问题三保存结果的独立快速复核。

先用线段包围盒与圆包围盒作常数时间筛查，仅对可能相交的航段求解析交区间；
再逐事件核验飞行、服务和等待均不与正时长管制区冲突。
"""

from __future__ import annotations

import argparse
import json
import math
from collections import Counter
from datetime import datetime
from pathlib import Path

import pandas as pd


REQ = {"I": 3, "II": 2, "III": 1}
TOL = 1.0e-7


def to_min(text: object) -> float:
    value = datetime.strptime(str(text).strip(), "%H:%M")
    return (value.hour - 8) * 60 + value.minute


def overlap(a0: float, a1: float, b0: float, b1: float) -> bool:
    return max(a0, b0) < min(a1, b1) - TOL
```

```python
def bbox_maybe(a, b, center, radius) -> bool:
    return not (
        max(a[0], b[0]) < center[0] - radius
        or min(a[0], b[0]) > center[0] + radius
        or max(a[1], b[1]) < center[1] - radius
        or min(a[1], b[1]) > center[1] + radius
    )


def segment_inside_interval(a, b, center, radius):
    if not bbox_maybe(a, b, center, radius):
        return None
    dx, dy = b[0] - a[0], b[1] - a[1]
    fx, fy = a[0] - center[0], a[1] - center[1]
    aa = dx * dx + dy * dy
    bb = 2 * (fx * dx + fy * dy)
    cc = fx * fx + fy * fy - radius * radius
    disc = bb * bb - 4 * aa * cc
    if disc < -TOL:
        return None
    disc = max(0.0, disc)
    lo = max(0.0, (-bb - math.sqrt(disc)) / (2 * aa))
    hi = min(1.0, (-bb + math.sqrt(disc)) / (2 * aa))
    return None if lo > hi + TOL else (lo, hi)


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--solution", type=Path, required=True)
    args = parser.parse_args()

    result = json.loads(args.solution.read_text(encoding="utf-8"))
    points_book = pd.ExcelFile(args.points)
    zones_book = pd.ExcelFile(args.zones)
    assert set(result) == set(points_book.sheet_names) ==
    ↪  set(zones_book.sheet_names)

    for case_name in points_book.sheet_names:
        points_df = pd.read_excel(points_book, sheet_name=case_name)
        zones_df = pd.read_excel(zones_book, sheet_name=case_name)
        coords = {0: (0.0, 0.0)}
        expected = {}
        for _, row in points_df.iterrows():
            point_id = int(row.Point_ID)
            coords[point_id] = (float(row.X_Coordinate),
            ↪  float(row.Y_Coordinate))
            expected[point_id] = REQ[str(row.Inspection_Level)]
        zones = []
        for _, row in zones_df.iterrows():
            start, end = to_min(row.Start_Time), to_min(row.End_Time)
            if end > start + TOL:
                zones.append((str(row.Zone_ID), (float(row.Center_X),
                ↪  float(row.Center_Y)), float(row.Radius), start,
                ↪  end))

        found = Counter()
        times = []
        for route in result[case_name]["routes"]:
            seq = [int(x) for x in route["sequence"]]
            assert seq[0] == seq[-1] == 0
            assert all(a != b for a, b in zip(seq[1:-1], seq[2:-1]))
            found.update(seq[1:-1])
            last_end = 0.0
            for event in route["events"]:
                start, end = float(event["start_min"]),
                ↪  float(event["end_min"])
                assert abs(start - last_end) < 2.0e-6
                assert end >= start - TOL
                if event["type"] == "flight":
                    a, b = coords[int(event["from"])],
                    ↪  coords[int(event["to"])]
                    for zone_id, center, radius, z0, z1 in zones:
                        interval = segment_inside_interval(a, b,
                        ↪  center, radius)
                        if interval is None:
                            continue
                        lo, hi = interval
                        assert not overlap(start + lo * (end -
                        ↪  start), start + hi * (end - start), z0,
                        ↪  z1), f"{case_name} UAV{route['uav']}
                        ↪  flight conflicts {zone_id}"
                elif event["type"] in {"service", "wait"}:
                    node = int(event["node"])
                    if event["type"] == "wait" and node == 0:
                        last_end = end
                        continue
                    point = coords[node]
                    for zone_id, center, radius, z0, z1 in zones:
                        if math.dist(point, center) <= radius + TOL:
                            assert not overlap(start, end, z0, z1),
                            ↪  f"{case_name} UAV{route['uav']}
                            ↪  {event['type']} conflicts {zone_id}"
                last_end = end
            assert abs(last_end / 60 - float(route["time_h"])) <
            ↪  2.0e-6
            times.append(last_end / 60)
        assert dict(found) == expected
        case = result[case_name]
        assert int(case["N"]) == len(case["routes"])
        assert abs(max(times) - float(case["Tmax"])) < 2.0e-6
        assert abs(min(times) - float(case["Tmin"])) < 2.0e-6
        assert abs(max(times) - min(times) - float(case["delta"])) <
        ↪  2.0e-6
        print(f"{case_name}: PASSED")


if __name__ == "__main__":
    main()
```

问题三算法 B 逐事件独立验证程序

路径：submission_package/question3/program/q3_enhanced_validator.py

```python
""" 问题三增强算法的独立逐事件验证器。"""

from __future__ import annotations

import argparse
import json
from collections import Counter
from pathlib import Path

import q3_solver as baseline
from q3_solver_enhanced import expected_copy_ids, task_point


TOL = 2.0e-6


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--points", type=Path, required=True)
    parser.add_argument("--zones", type=Path, required=True)
    parser.add_argument("--solution", type=Path, required=True)
    args = parser.parse_args()

    cases = baseline.load_cases(args.points, args.zones)
    result = json.loads(args.solution.read_text(encoding="utf-8"))
    assert set(result) == set(cases)
    for case_name, scenarios in result.items():
        data = cases[case_name]
        expected = expected_copy_ids(data)
        for scenario_name, case in scenarios.items():
            found_tasks: list[str] = []
            times: list[float] = []
            total_wait = 0.0
            for route in case["routes"]:
                task_sequence = route["task_sequence"]
                assert task_sequence[0] == task_sequence[-1] ==
                ↪  "BASE"
                tasks = task_sequence[1:-1]
                found_tasks.extend(tasks)
```

```
            physical = [task_point(task) for task in tasks]
            assert route["sequence"] == [0, *physical, 0]
            assert all(a != b for a, b in zip(physical,
            ↪  physical[1:]))

            last_end = 0.0
            services: list[str] = []
            distance_km = 0.0
            for event in route["events"]:
                start = float(event["start_min"])
                end = float(event["end_min"])
                assert abs(start - last_end) < TOL
                assert end >= start - TOL
                if event["type"] == "flight":
                    a = tuple(float(x) for x in event["from_xy"])
                    b = tuple(float(x) for x in event["to_xy"])
                    distance_km += float(event["distance_km"])
                    for zone in data.zones:
                        if not zone.active:
                            continue
                        interval =
                        ↪  baseline.segment_circle_interval(a,
                        ↪  b, zone)
                        if interval is None:
                            continue
                        lo, hi = interval
                        assert not baseline.overlaps(
                            start + lo * (end - start),
                            start + hi * (end - start),
                            zone.start_min,
                            zone.end_min,
                        ), f"{case_name}/{scenario_name}/UAV{rou
                        ↪  te['uav']} flight conflicts
                        ↪  {zone.zone_id}"
                elif event["type"] in {"wait", "service"}:
                    xy = tuple(float(x) for x in
                    ↪  event["node_xy"])
                    if event["type"] == "wait":
                        total_wait += end - start
                    else:
                        services.append(event["task_id"])
                        assert abs((end - start) -
                        ↪  baseline.SERVICE_MIN) < TOL
                    for zone in data.zones:
                        if zone.active and
                        ↪  baseline.point_inside(xy, zone):
                            assert not baseline.overlaps(start,
                            ↪  end, zone.start_min,
                            ↪  zone.end_min), (
                                f"{case_name}/{scenario_name}/UA
                                ↪  V{route['uav']} "
                                f"{event['type']} conflicts
                                ↪  {zone.zone_id}"
                            )
                else:
                    raise AssertionError(f"未知事件类型
                    ↪  {event['type']}")
                last_end = end
            assert services == tasks
            assert abs(distance_km - float(route["distance_km"]))
            ↪  < TOL
            assert abs(last_end / 60.0 - float(route["time_h"]))
            ↪  < TOL
            times.append(float(route["time_h"]))

        assert len(found_tasks) == len(set(found_tasks))
        assert set(found_tasks) == expected
        assert int(case["N"]) == len(case["routes"])
        assert abs(max(times) - float(case["Tmax"])) < TOL
        assert abs(min(times) - float(case["Tmin"])) < TOL
        assert abs(max(times) - min(times) -
        ↪  float(case["delta"])) < TOL
        assert abs(total_wait - float(case["total_wait_min"])) <
        ↪  TOL
        limit = float(case["stage_b"]["Tmax_limit_h"])
        assert float(case["Tmax"]) <= limit + TOL
        assert case["deadline_17_00"] is False
        print(f"{case_name}/{scenario_name}: PASSED")


if __name__ == "__main__":
    main()
```